

**DURWANKUR TECHNOLOGIES PVT. LTD.**

*Platform Provider*

**CCTV AMC & Service Management SaaS  
Platform**

**Technical Architecture Document (TAD)**

*Multi-Tenant SaaS Platform Architecture for CCTV Sales, Installation, AMC & Service  
Operations*

Prepared For: Durwankur Technologies Pvt. Ltd.  
Prepared By: Senior Enterprise Solution Architecture Team  
Document Version: 1.0 | Status: Draft for Review  
Document Date: 20 June 2026

*Reference: CCTV AMC & Service Management SaaS Platform – Software Requirement Specification v1.2*

## Document Control

### Document Information

| Field          | Details                                                                          |
|----------------|----------------------------------------------------------------------------------|
| Document Title | CCTV AMC & Service Management SaaS Platform – Technical Architecture Document    |
| Prepared For   | Durwankur Technologies Pvt. Ltd. (Platform Provider)                             |
| Prepared By    | Senior Enterprise Solution Architect / Technical Architecture Team               |
| Source SRS     | CCTV AMC & Service Management SaaS Platform SRS, Version 1.2, dated 20 June 2026 |
| Version        | 1.0                                                                              |
| Status         | Draft for Review                                                                 |
| Document Date  | 20 June 2026                                                                     |

### Revision History

| Version | Date         | Description                                                    | Author            |
|---------|--------------|----------------------------------------------------------------|-------------------|
| 1.0     | 20 June 2026 | Initial Technical Architecture Document derived from SRS v1.2. | Architecture Team |

### Intended Audience

- Development Team – backend, frontend, and mobile engineers implementing the platform.
- Technical Leads – module owners responsible for design decisions and code quality.
- DevOps Team – responsible for CI/CD, containerization, and infrastructure operations.
- Database Architects – responsible for schema design, indexing, and performance tuning.
- QA Team – responsible for functional, non-functional, and multi-tenant isolation testing.
- Project Managers – responsible for planning, tracking, and delivery of the program.
- Infrastructure Team – responsible for provisioning, monitoring, and capacity planning.
- Client / Stakeholder Review Committee – for architecture review and sign-off.

## Table of Contents

---

# 1. Executive Summary

## 1.1 Project Overview

Durwankur Technologies Pvt. Ltd. is building the CCTV AMC & Service Management SaaS Platform: a true multi-tenant, cloud-hosted software product that independent CCTV sales, installation, and Annual Maintenance Contract (AMC) service-provider companies (“Tenants”) subscribe to and use to run their end-to-end operations. A single deployment of the platform serves many Tenants concurrently, with each Tenant's leads, customers, vendors, sites, assets, contracts, tickets, inventory, quotations, invoices, and users held in logically isolated, Tenant-scoped data.

This Technical Architecture Document (TAD) translates the functional and non-functional requirements captured in the CCTV AMC & Service Management SaaS Platform SRS (v1.2) into a concrete, build-ready architecture: technology choices, layering, module decomposition, data model, API design, security model, deployment topology, and operational standards. It is the single technical reference that the development, QA, DevOps, database, and infrastructure teams use to design, build, test, and operate the system, and that Durwankur Technologies and reviewing stakeholders use to approve the proposed architecture before implementation begins.

## 1.2 Business Objectives

- **Multi-tenant SaaS monetization:** enable Durwankur Technologies to onboard and bill multiple independent CCTV AMC service-provider companies on one shared platform, with strict per-Tenant data isolation and configurable subscription plans.
- **Operational digitization for Tenants:** replace spreadsheet- and paper-based tracking of leads, AMC contracts, service tickets, technician visits, inventory, and billing with a single connected system.
- **Faster sales-to-cash cycle:** shorten the path from lead capture through quotation, installation, AMC sign-up, invoicing, and payment collection.
- **Service quality and SLA discipline:** give Tenants ticket SLA tracking, technician dispatch, and preventive-maintenance scheduling to reduce missed visits and resolution delays.
- **Operational visibility:** provide management dashboards and reports covering lead pipeline, AMC renewals, ticket SLAs, technician productivity, inventory health, and revenue.
- **Platform scalability:** support Tenant growth and new Tenant onboarding without downtime or rework to existing Tenants' deployments.

## 1.3 Key System Capabilities

| Capability Area                  | What It Delivers                                                                                                                                                   |
|----------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Platform & Tenant administration | Tenant onboarding, branding/white-labeling, subscription plans, usage limits, lifecycle status, and platform-wide monitoring for Durwankur Technologies.           |
| Sales lifecycle                  | Lead capture and qualification, conversion to Customer Master and Quotation, and conversion of approved quotations into AMC contracts or installation work orders. |
| Customer & asset management      | Unified Customer Master for CHS, Commercial, and Single Shop categories; multi-site and CCTV asset registers with warranty tracking.                               |
| AMC & service operations         | AMC contract lifecycle, preventive-maintenance scheduling, ticketing with                                                                                          |

| Capability Area       | What It Delivers                                                                                                                                           |
|-----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                       | SLA timers, and technician dispatch with GPS-verified visits.                                                                                              |
| Commerce & finance    | Quotations, sales orders, GST-compliant invoicing (including recurring AMC billing), payment and ageing tracking, and vendor payables.                     |
| Inventory             | Centralized and warehouse/van-stock inventory with auto-deduction on consumption and vendor-linked reorder workflows.                                      |
| Engagement            | Multi-channel notifications (Email, SMS, WhatsApp, in-app) for expiries, dues, escalations, and reminders.                                                 |
| Insight               | Role-based dashboards and exportable reports across the sales, service, inventory, and finance lifecycle.                                                  |
| Mobile & self-service | Offline-capable React Native app for field technicians, and a customer self-service portal for complaint tracking, quotation approval, and invoice access. |

## 1.4 High-Level Solution Architecture

The platform is delivered as four client surfaces (Platform Admin console, Tenant back-office web portal, customer self-service portal, and a React Native mobile app) on a shared, multi-tenant backend. All client traffic is terminated at an Nginx reverse proxy, routed to a horizontally scalable FastAPI application tier, and persisted to a PostgreSQL database in which every Tenant-owned table carries a `tenant_id` and is filtered by a tenant-context middleware on every request. Background processing (notifications, scheduled jobs, recurring billing) runs on a separate worker tier backed by Redis, decoupled from the synchronous request path. The complete diagram for this architecture is presented as Figure 1 in Section 2.1, and is elaborated layer-by-layer in Sections 2 through 4.

The architecture is deliberately conventional and operationally boring by design: a layered, container-based, REST/JSON system using mainstream, well-supported technologies (React, FastAPI, PostgreSQL, Docker, Nginx) rather than bespoke infrastructure. This minimizes hiring and onboarding risk for the development and DevOps teams, keeps the operational surface area small for a platform that must run reliably for many paying Tenants, and leaves room to scale horizontally as Tenant count grows, per the SRS scalability requirements (Section 5.2 of the SRS).

## 2. System Architecture Overview

### 2.1 High-Level Architecture Diagram

Figure 1 shows the full system context: the four client surfaces, the edge/gateway layer, the Dockerized application tier, the data tier, and the external integrations the platform depends on (email/SMS/WhatsApp gateways and payment gateway hooks).

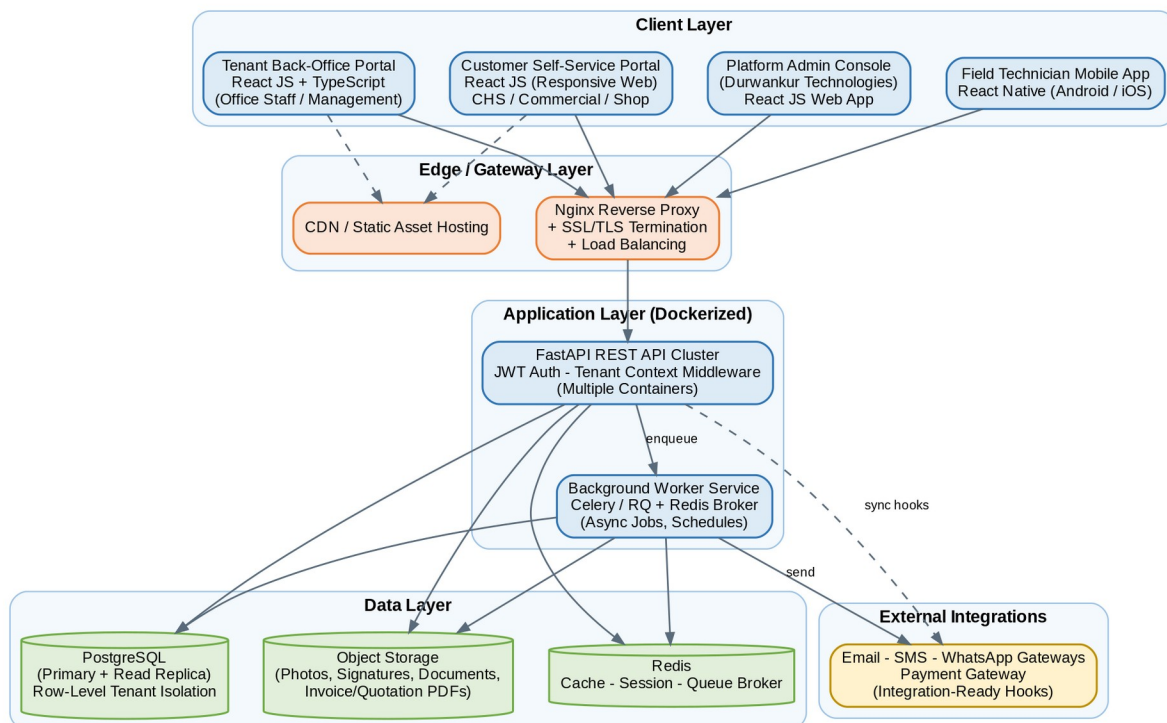


Figure 1: High-Level Architecture – System Context

Key points of this view:

- All four client surfaces (Platform Admin, Tenant portal, Customer portal, Mobile app) talk to the same FastAPI backend through one Nginx entry point — there is no separate backend per client.
- Static assets for the React web apps are served from a CDN; the Nginx/API path only ever carries JSON traffic and file uploads/downloads.
- Background work (notification dispatch, recurring AMC billing, scheduled reports) is pushed onto a worker tier through Redis so the request/response path stays fast.
- PostgreSQL is the single source of truth; an object store holds binary content (photos, signatures, generated PDFs, warranty documents) referenced by URL/key from PostgreSQL rows.

### 2.2 Layered Architecture

The backend is organized into six strict layers. Each layer only calls the layer directly beneath it, which keeps business logic independent of transport concerns and keeps the data-access code

independent of business rules — both are required for the platform to support automated multi-tenant isolation testing called out in the SRS (Section 5.3).

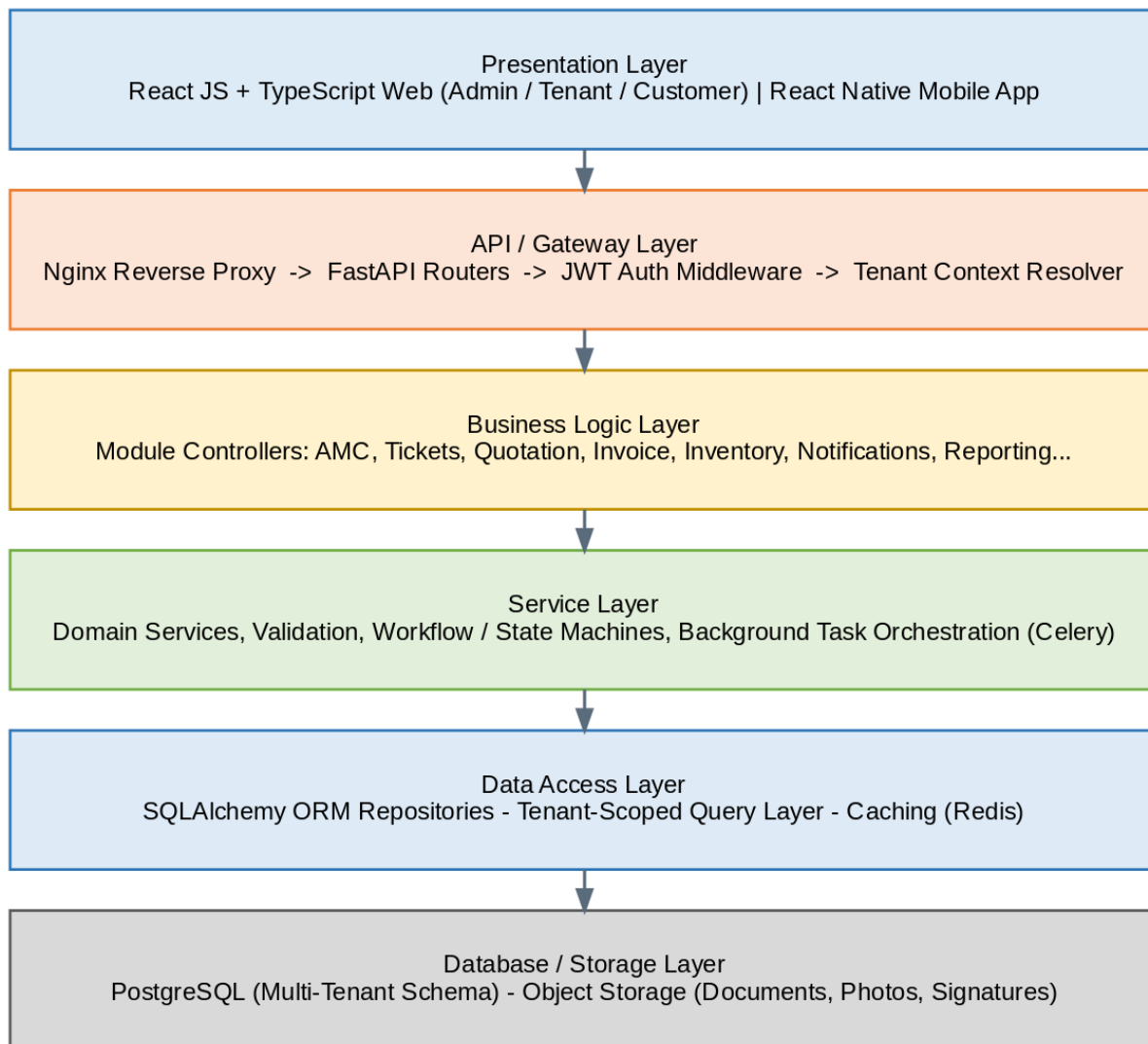


Figure 2: Layered Architecture

| Layer          | Responsibility                                                                                 | Primary Technology                     |
|----------------|------------------------------------------------------------------------------------------------|----------------------------------------|
| Presentation   | Renders UI, manages client-side state, calls REST APIs.                                        | React JS + TypeScript, React Native    |
| API / Gateway  | TLS termination, routing, request validation, authentication, tenant resolution.               | Nginx, FastAPI routers, JWT middleware |
| Business Logic | Module-level rules: lead conversion, AMC lifecycle, ticket SLA, quotation-to-order conversion. | FastAPI controllers / module packages  |
| Service        | Cross-cutting domain services, workflow/state machines, orchestration of background tasks.     | Python service classes, Celery tasks   |
| Data Access    | Tenant-scoped repositories, query building, caching.                                           | SQLAlchemy ORM, Redis cache            |

| Layer              | Responsibility                       | Primary Technology         |
|--------------------|--------------------------------------|----------------------------|
| Database / Storage | System of record and binary storage. | PostgreSQL, object storage |

## 2.3 Component-Based Architecture

Within the Business Logic and Service layers, the system is decomposed into independently testable modules, each owning its own database entities and exposing its own versioned API routes behind the shared gateway. Modules call one another only through well-defined service interfaces (never directly against another module's tables), which keeps the codebase modular today and leaves a path to splitting hot modules into separate deployable services later if scale requires it.

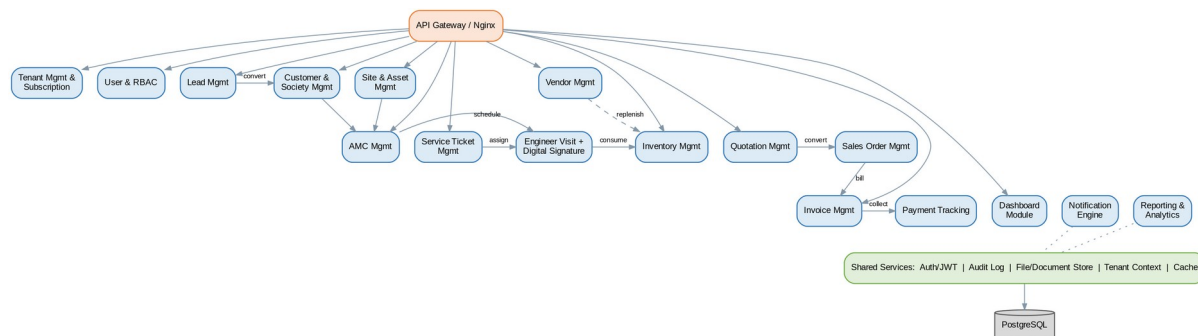


Figure 3: Component-Based Architecture – Module Map

Arrows above show the primary business hand-offs: Lead → Customer, Quotation → Sales Order, Sales Order → Invoice, Invoice → Payment, AMC/Ticket → Engineer Visit, and Visit → Inventory consumption. Notification Engine and Reporting & Analytics are cross-cutting modules that read from, but do not own, other modules' data.

## 2.4 Logical Architecture Diagram

Logically, the platform is one application instance shared by all Tenants. Multi-tenancy is implemented at the row level: every Tenant-owned table carries a `tenant_id` column, and a tenant-context middleware (populated from the authenticated user's JWT) is injected into every ORM query so that no code path can read or write another Tenant's rows. This is the pooled/shared-schema multi-tenancy model — chosen over schema-per-Tenant or database-per-Tenant because it gives the lowest operational overhead while still meeting the SRS's strict logical-isolation requirement (Section 5.3), and it scales to hundreds of Tenants without a proliferation of schemas or databases to migrate and monitor.



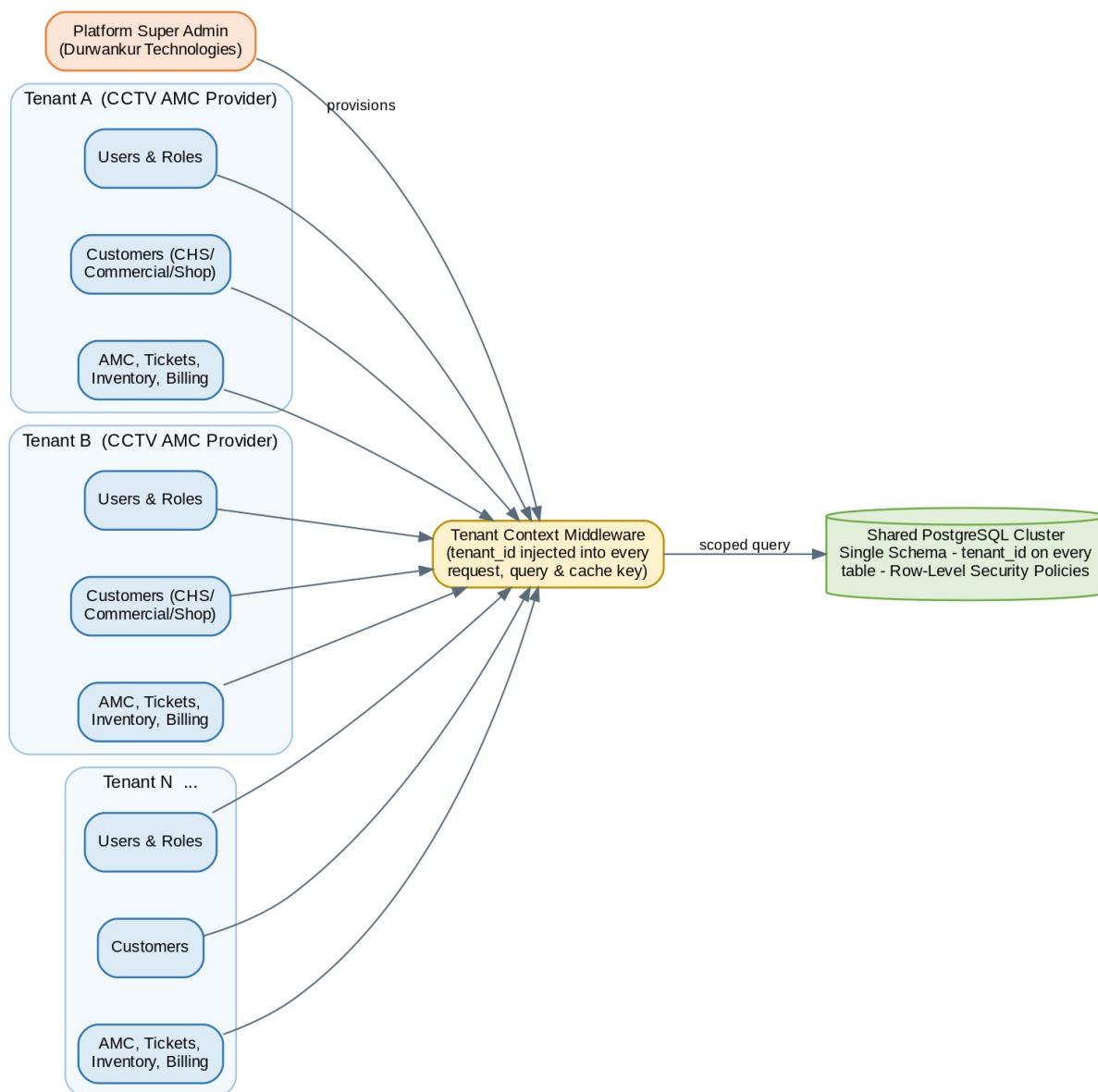


Figure 4: Logical Architecture – Multi-Tenant Isolation Model

- **Tenant context propagation:** the JWT issued at login embeds `tenant_id` (and, for Platform Super Admin, a platform-scope flag); the tenant-context middleware reads this once per request and attaches it to the request-scoped database session.
- **Enforcement points:** `tenant_id` filtering is enforced in the repository layer (defense in depth #1) and reinforced with PostgreSQL Row-Level Security policies (defense in depth #2), so a missed filter in application code is still blocked at the database.
- **Platform Super Admin:** is the only role that can query across Tenants, and only through dedicated platform-admin endpoints that are audited separately from Tenant-scoped endpoints.

## 2.5 Physical Architecture Diagram

Physically, the platform runs as Dockerized services across two or more Ubuntu Linux nodes behind a cloud load balancer, with a managed PostgreSQL primary/replica pair, a Redis cluster for cache/session/queue, and object storage for documents and media. Section 12 (Deployment Architecture) details environment-specific topology (Dev/UAT/Prod); the diagram below shows the steady-state production topology.

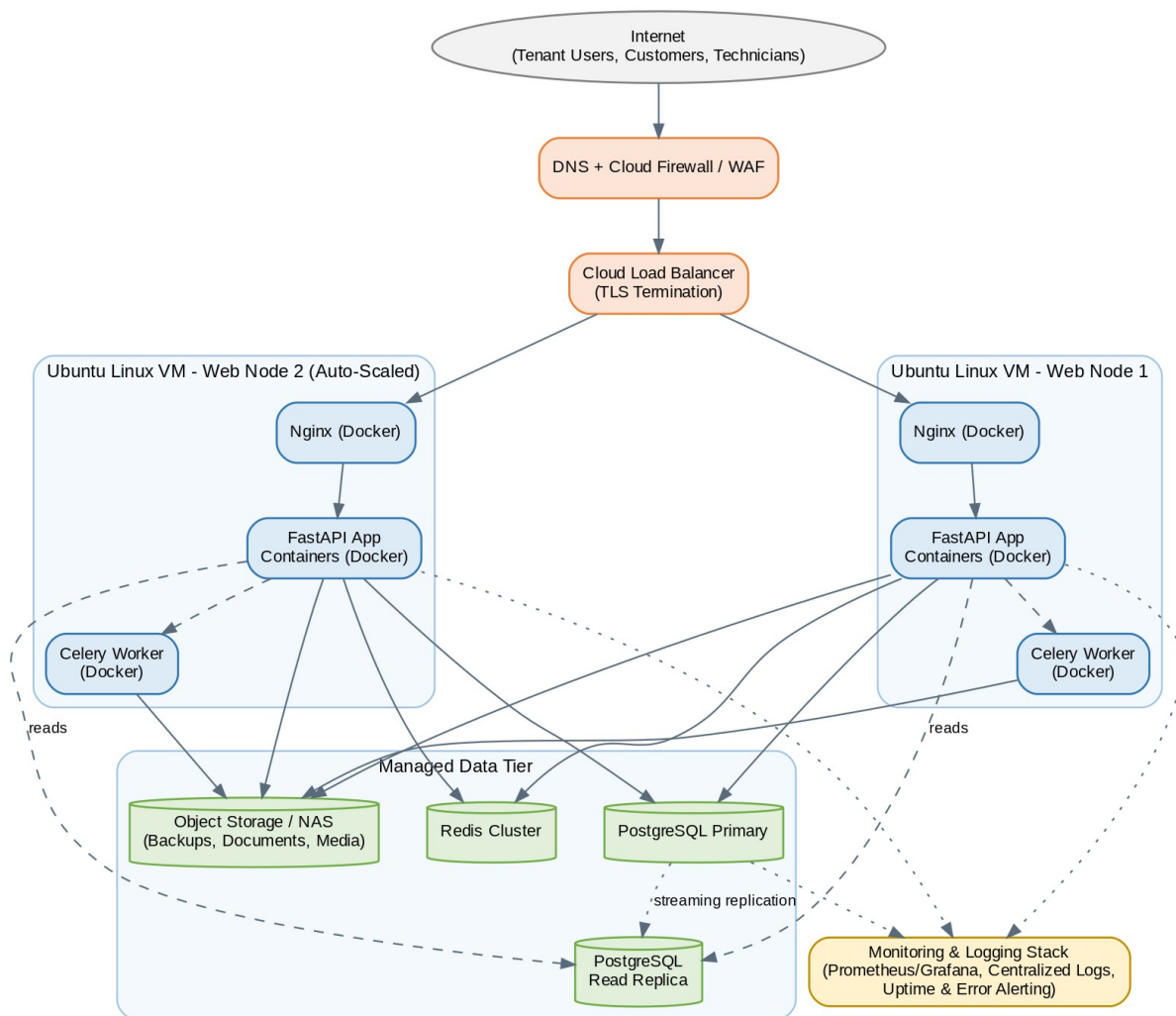


Figure 5: Physical / Deployment Architecture – Production Topology

Each web node runs three containers — Nginx, the FastAPI application, and a Celery worker — so that nodes can be added or removed as a unit during horizontal scaling (Section 13). The PostgreSQL primary handles all writes; the read replica offloads reporting and dashboard queries so they cannot contend with transactional traffic.

### 3. Technology Architecture

This section justifies each technology selection against the platform's actual requirements: multi-tenant SaaS delivery, a field-mobile workforce, GST-compliant billing, and the SRS's performance, scalability, and security targets (Sections 5.1–5.6 of the SRS). Each subsection covers rationale, benefits, scalability, security, and performance considerations.

#### 3.1 React JS + TypeScript (Web Frontend)

| Dimension   | Assessment                                                                                                                                                                                                                                                                                                                                                                   |
|-------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Rationale   | Three distinct web surfaces (Platform Admin, Tenant back-office, Customer portal) share most UI patterns (tables, forms, dashboards, approvals). React's component model lets all three be built from one shared component library, and TypeScript's static typing catches integration errors against the FastAPI contract (via generated/typed API clients) before runtime. |
| Benefits    | Large ecosystem (Material UI / Ant Design component sets, Redux Toolkit, React Router), strong hiring pool, and component reuse across the three web apps reduces total frontend effort versus three separately-stacked apps.                                                                                                                                                |
| Scalability | Code-splitting and lazy-loaded routes per module (AMC, Tickets, Inventory, etc.) keep bundle size manageable as modules are added; the same component library scales to new Tenant-specific branding via theme tokens rather than forked code.                                                                                                                               |
| Security    | TypeScript reduces a class of runtime defects; all API calls are authenticated via JWT held in memory/secure storage (not exposed to XSS-readable localStorage for sensitive tokens); Content Security Policy and input sanitization are enforced at the component layer for any user-generated content (e.g., ticket comments).                                             |
| Performance | Server-rendered-free SPA with route-level code splitting, memoized table/list components for large datasets (customer lists, tickets), and Redux Toolkit Query for response caching/deduping to meet the SRS's 3-second page-load target (Section 5.1).                                                                                                                      |

#### 3.2 Material UI / Ant Design + Redux Toolkit + Axios

- **Material UI / Ant Design:** provide accessible, pre-built form, table, and dashboard components, letting the team focus engineering effort on AMC/ticketing/billing business logic rather than re-building basic UI primitives. One library is adopted platform-wide for visual consistency; Tenant white-labeling (Section 4.1 of the SRS) is achieved through theme tokens (colors, logo) layered on top, not component forks.
- **Redux Toolkit:** centralizes cross-cutting client state (current Tenant/branding context, authenticated user, RBAC permissions, notification counts) and, via RTK Query, manages server-state caching, request de-duplication, and optimistic updates for high-traffic screens such as ticket boards and dashboards.
- **Axios:** provides a single HTTP client with interceptors used for attaching JWT access tokens, transparent refresh-token rotation, standardized error normalization, and request/response logging in non-production environments.

### 3.3 FastAPI (Backend Framework)

| Dimension   | Assessment                                                                                                                                                                                                                                                                    |
|-------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Rationale   | FastAPI is a modern, async-capable Python web framework with native Pydantic-based request/response validation and automatic OpenAPI documentation generation — directly supporting the API Architecture's request/response and documentation standards (Section 7).          |
| Benefits    | Built-in dependency injection cleanly implements the JWT-auth → tenant-context → RBAC middleware chain (Figure 7); automatic OpenAPI/Swagger docs keep frontend, mobile, and QA teams working against a single live contract instead of hand-maintained API documents.        |
| Scalability | Async request handling allows a single worker process to serve many concurrent I/O-bound requests (typical for a CRUD-heavy admin/ops system); the stateless application containers scale horizontally behind Nginx/load balancer (Section 13).                               |
| Security    | Pydantic schemas reject malformed/unexpected payloads before they reach business logic; FastAPI's dependency system enforces authentication and RBAC checks consistently across every route, removing the risk of a route forgetting to check permissions.                    |
| Performance | Benchmarks consistently place FastAPI among the fastest Python frameworks; combined with async DB drivers and Redis caching for read-heavy endpoints (dashboards, customer lookups), it comfortably meets the SRS's 200-concurrent-user / 3-second page target (Section 5.1). |

### 3.4 JWT Authentication, REST API Architecture & Background Task Processing

- **JWT Authentication:** stateless, signed tokens let every API container verify a request without a shared session store, which is required for horizontal scaling of the FastAPI tier. Full strategy in Section 8.2.
- **REST API Architecture:** a uniform, resource-oriented, versioned REST/JSON API (Section 7) is used across web, mobile, and any future third-party integrations, avoiding the cost of maintaining multiple API styles.
- **Background Task Processing:** Celery (with Redis as broker) executes anything that should not block a request: notification dispatch, recurring AMC invoice generation, scheduled preventive-maintenance reminders, and report pre-aggregation. Celery Beat drives all cron-style schedules so scheduling logic lives in one place.

### 3.5 PostgreSQL (Database)

| Dimension | Assessment                                                                                                                                                                                                                                                                                                                                       |
|-----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Rationale | The domain is fundamentally relational (Tenants → Customers → Sites → Assets → Contracts → Tickets/Invoices, with strict referential integrity requirements for billing and GST compliance). PostgreSQL gives strong transactional guarantees plus Row-Level Security, which is used as the second layer of Tenant data isolation (Section 2.4). |
| Benefits  | Native JSONB for semi-structured data (notification templates, configurable AMC terms, custom role-permission sets), full-text search for customer/ticket search, and a mature ecosystem of operational tooling (pg_dump, logical replication, pg_stat_statements).                                                                              |

| Dimension   | Assessment                                                                                                                                                                                                                                                         |
|-------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Scalability | Read replicas offload reporting/dashboard load from the transactional primary (Figure 5); partitioning of high-growth tables (audit log, stock ledger, notifications) by tenant_id or date keeps query plans efficient as data volume grows (Section 6.7).         |
| Security    | Row-Level Security policies enforce Tenant isolation at the database engine itself; column-level encryption for sensitive fields (bank details, GSTIN where required); all connections require TLS.                                                                |
| Performance | Composite and partial indexes on (tenant_id, status), (tenant_id, created_at), and similar predicates keep the platform's most common queries — "my Tenant's open tickets", "my Tenant's expiring AMC contracts" — index-only scans even as the Tenant base grows. |

### 3.6 React Native (Mobile Application)

| Dimension   | Assessment                                                                                                                                                                                                                                                                      |
|-------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Rationale   | A single React Native codebase delivers both Android and iOS apps for field technicians (SRS Section 4.18), sharing business logic and a large fraction of UI code with the React web stack, which reduces the specialist hiring need versus separate native Android/iOS teams. |
| Benefits    | Mature offline-storage and background-sync libraries; large plugin ecosystem for camera, GPS, and signature capture (Section 10); over-the-air update channels (e.g., CodePush-style) for non-store-review fixes.                                                               |
| Scalability | The same app and API surface serve a growing technician headcount across all Tenants; per-Tenant feature flags (rather than per-Tenant app builds) keep one app binary maintainable. White-labeled per-Tenant apps are explicitly Phase 2 (SRS Section 7).                      |
| Security    | JWT stored in encrypted device storage (Keychain/Keystore-backed secure storage), certificate pinning for API calls, and remote session revocation on device loss or staff offboarding.                                                                                         |
| Performance | Local SQLite-backed offline store (Section 10.3) means ticket/visit screens render instantly regardless of network quality, with sync occurring in the background — directly satisfying the SRS's offline-operation requirement (Section 5.4).                                  |

### 3.7 Docker (Containerization)

- **Rationale:** every backend service (API, worker, and supporting tooling) ships as an immutable, versioned image, eliminating "works on my machine" drift between developer, CI, UAT, and production environments.
- **Benefits:** identical images run unmodified across Dev, UAT, and Production (Section 12); rollback is a one-command image-tag revert.
- **Scalability:** containers are the horizontal-scaling unit — adding capacity is adding container replicas behind the load balancer, with no host-level reconfiguration.
- **Security:** minimal base images (slim/distroless where practical), no secrets baked into images (injected via environment/secret store at runtime), and images are scanned for known vulnerabilities in CI (Section 16.2).
- **Performance:** fast container start-up supports quick auto-scaling and rolling deploys with minimal capacity dip during release.

### 3.8 Nginx (Reverse Proxy)

- **Rationale:** a single, well-understood edge layer handles TLS termination, request routing, gzip compression, and static-asset caching in front of the FastAPI containers.
- **Benefits:** low resource footprint, mature load-balancing algorithms (round-robin/least-connections) across API container replicas, and straightforward rate-limiting configuration.
- **Scalability:** Nginx itself is replicated behind the cloud load balancer (Figure 5), so the edge layer scales the same way as the application tier.
- **Security:** enforces TLS 1.2+ only, applies security headers (HSTS, X-Content-Type-Options, X-Frame-Options), and provides the first layer of rate-limiting/brute-force mitigation ahead of the application.
- **Performance:** serves cached static assets and compresses responses, reducing backend load and improving perceived page performance.

### 3.9 Technology Stack Recommendation Summary

This table is the platform's authoritative technology baseline; any deviation during implementation should be raised and re-approved through architecture review rather than substituted silently.

| Layer                   | Recommended Technology                                               | Notes                                                                       |
|-------------------------|----------------------------------------------------------------------|-----------------------------------------------------------------------------|
| Web Frontend            | React JS, TypeScript, Material UI / Ant Design, Redux Toolkit, Axios | Single component library shared across Admin, Tenant, and Customer portals. |
| Mobile App              | React Native (Android & iOS)                                         | Offline-first, shared business logic with web.                              |
| Backend API             | Python, FastAPI, Pydantic                                            | Async, OpenAPI-documented, JWT-secured REST API.                            |
| Background Processing   | Celery + Redis (broker)                                              | Notifications, recurring billing, scheduled jobs.                           |
| Database                | PostgreSQL (primary + read replica)                                  | Row-Level Security for Tenant isolation.                                    |
| Cache / Session / Queue | Redis                                                                | API response cache, rate-limit counters, Celery broker.                     |
| Object Storage          | S3-compatible object storage                                         | Photos, signatures, generated PDFs, documents.                              |
| Containerization        | Docker                                                               | All services packaged as versioned images.                                  |
| Reverse Proxy / Edge    | Nginx                                                                | TLS termination, routing, load balancing.                                   |
| OS / Hosting            | Ubuntu Linux Server, Cloud Hosting Environment                       | Cloud-agnostic; sized per Section 12.                                       |
| CI/CD                   | Git + CI/CD pipeline (Section 16)                                    | Automated build, test, scan, deploy.                                        |
| Monitoring              | Prometheus/Grafana-class stack + centralized logging                 | Section 16.3.                                                               |

## 4. Application Architecture

This section drills into each layer introduced in Figure 2 (Section 2.2), describing its internal structure, conventions, and the contract it exposes to the layer above. Module-specific detail (entities, APIs, roles per business module) follows in Section 5.

### 4.1 Frontend Layer

Three React + TypeScript single-page applications share one component and state-management foundation, differentiated by routing, RBAC-driven menu visibility, and Tenant theming rather than by separate codebases:

| App                          | Primary Users                                                                                                                 | Distinguishing Behavior                                                                          |
|------------------------------|-------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| Platform Admin Console       | Platform Super Admin (Durwankur Technologies)                                                                                 | Cross-Tenant views: Tenant list, subscription/billing, usage metrics, platform health.           |
| Tenant Back-Office Portal    | Tenant Admin, Sales Executive, Operations Manager, Service Coordinator, Accounts/Billing, Procurement/Vendor Manager, Auditor | Tenant-scoped only; menu and actions rendered from the user's RBAC permission set.               |
| Customer Self-Service Portal | Society/Client Admin (CHS, Commercial, Single Shop)                                                                           | Mobile-responsive, no app install required (SRS 5.6); scoped to the customer's own records only. |

- **Folder convention:** feature-first structure (features/amc, features/tickets, features/inventory, ...), each owning its own Redux slice, API hooks, and route components.
- **State management:** Redux Toolkit slices for cross-cutting state (auth/session, Tenant theme, notifications) plus RTK Query for all server data, giving automatic caching, polling, and invalidation on mutation.
- **API communication:** Axios instance per app with interceptors for JWT attachment, silent refresh-token rotation, and centralized error-to-toast mapping.
- **Theming / white-labeling:** a Tenant theme object (logo, colors, document templates) is fetched on login and applied at the Material UI / Ant Design theme-provider level — no per-Tenant builds.

### 4.2 API Layer

Implemented as FastAPI routers, one per business module (Section 5), mounted under a versioned prefix (/api/v1/...). Each router only depends on the Service layer below it — never directly on the ORM — so the API layer's job is strictly request parsing, response shaping, and delegating to services.

```

app/
  api/v1/
    auth.py          # login, refresh, logout
    tenants.py       # platform-admin tenant management
    customers.py
    leads.py
    amc_contracts.py
    tickets.py
    visits.py

```



```
inventory.py
quotations.py
sales_orders.py
invoices.py
payments.py
notifications.py
reports.py
core/          # security, settings, tenant context
services/      # business logic per module
repositories/  # data access per module
models/        # SQLAlchemy models
schemas/       # Pydantic request/response schemas
workers/       # Celery tasks
```

### 4.3 Business Logic Layer

Each module's router calls a corresponding Service class that encodes the SRS's business rules — for example, the AMC service enforces that an AMC contract can only be created from an approved quotation or installation handover, and the Ticket service enforces SLA-timer assignment based on priority at creation time. This layer is the one place these rules are encoded; routers and repositories never re-implement them.

### 4.4 Service Layer

Below the per-module business logic sits a thinner Service layer of cross-cutting domain services consumed by multiple modules:

- Tenant Context Service – resolves and validates the current Tenant/user/role for every request.
- Numbering Service – generates Tenant-scoped sequential numbers for tickets, quotations, sales orders, and invoices.
- Workflow / State-Machine Service – enforces valid status transitions (e.g., Quotation: Draft → Sent → Viewed → Approved/Rejected/Expired).
- Notification Dispatch Service – the single entry point any module calls to trigger an Email/SMS/WhatsApp/in-app notification (Section 9).
- Document/PDF Generation Service – renders quotation, invoice, and warranty PDFs from Tenant-specific templates.
- Audit Service – writes the tamper-evident audit log entry for every create/update/delete/approval action (Section 8.6).

### 4.5 Data Access Layer

SQLAlchemy repositories expose typed, intention-revealing methods (e.g., `get_open_tickets_for_tenant(tenant_id)`, `get_expiring_amc_contracts(tenant_id, within_days)`) rather than leaking raw query-building into services. Every repository method requires a `tenant_id` (or an explicitly-flagged platform-scope call) as a matter of interface design, so omitting Tenant scoping is a compile-time/code-review-visible mistake, not a silent runtime one. Frequently-read, slow-changing data (Tenant branding/config, role-permission matrices) is cached in Redis with explicit invalidation on write.



## 4.6 Database Layer

PostgreSQL is the system of record for all structured data; see Section 6 for the full schema design, ER diagram, indexing, and optimization strategy. Binary content (photos, signatures, generated documents) is never stored in PostgreSQL — only its object-storage key/URL is.

## 4.7 Mobile Application Layer

The React Native app is architecturally a fifth client on the same API layer, with one addition: an offline-sync layer between the UI and the API client (full detail in Section 10). It consumes the same `/api/v1/...` contract as the web apps, scoped to a mobile-relevant subset of endpoints (assigned tickets/visits, installation jobs, inventory consumption, signature/photo upload).

## 5. Module-Wise Architecture

This section documents every business module the platform must implement. The functional list requested for this TAD (User Management, Customer & Society Management, AMC Management, Service Ticket Management, Engineer Visit Management, Digital Signature Capture, Inventory Management, Quotation Management, Sales Order Management, Invoice Management, Payment Tracking, Notification Engine, Reporting & Analytics, Dashboard Module, Mobile App Module) is fully covered below. Three additional modules — Tenant & Platform Administration, Lead Management, and Vendor Management — are included because the source SRS defines the platform as a multi-tenant SaaS product with a sales pipeline and vendor-linked procurement; without them, AMC, Quotation, Sales Order, and Inventory cannot function as specified. Each module is documented with its functional description, database entities, required APIs, user roles, and integration requirements.

### 5.1 Tenant & Platform Administration Management

#### Functional Description

Allows Durwankur Technologies (Platform Super Admin) to onboard, configure, brand, subscribe, and monitor every CCTV AMC & service-provider company (Tenant) running on the platform. Included here because every other module in this section operates inside the Tenant boundary this module creates and enforces (SRS Section 4.1).

- Tracks Tenant lifecycle: Trial → Active → Suspended → Cancelled, with plan upgrade/downgrade/renewal.
- Generates platform-level subscription invoices to Tenants, distinct from the Tenant's own customer invoicing (Section 5.13).

#### Database Entities

| Entity              | Key Attributes                                            | Notes                                   |
|---------------------|-----------------------------------------------------------|-----------------------------------------|
| tenant              | id, name, gstin, logo_url, theme_json, status, created_at | Root of all Tenant-scoped data.         |
| subscription_plan   | id, name, max_users, max_sites, max_storage_gb, price     | Starter / Growth / Enterprise.          |
| tenant_subscription | id, tenant_id, plan_id, start_date, end_date, status      | Current and historical plan assignment. |
| platform_invoice    | id, tenant_id, period, amount, status                     | Durwankur → Tenant billing.             |

#### APIs Required

| Method & Path                              | Purpose                                                   |
|--------------------------------------------|-----------------------------------------------------------|
| POST /api/v1/platform/tenants              | Onboard a new Tenant with profile, plan, and branding.    |
| GET /api/v1/platform/tenants               | List all Tenants with usage/status (Platform Admin only). |
| PATCH /api/v1/platform/tenants/{id}/status | Suspend, reactivate, or cancel a Tenant.                  |
| PATCH /api/v1/platform/tenants/{id}/plan   | Upgrade/downgrade subscription plan.                      |

| Method & Path                           | Purpose                                            |
|-----------------------------------------|----------------------------------------------------|
| GET /api/v1/platform/tenants/{id}/usage | Usage metrics against plan limits.                 |
| GET /api/v1/tenant/profile              | Tenant Admin: read/update own branding and config. |

### User Roles

- Platform Super Admin – full access, all Tenants.
- Tenant Admin – read/update own Tenant profile and branding only.

### Integration Requirements

- Billing/payment gateway for platform-level subscription invoices.
- Email/SMS for Tenant onboarding and plan-expiry notifications.

## 5.2 User Management (Roles & Permissions)

### Functional Description

Implements the two-tier role model required by the SRS (Section 4.20): platform-level roles scoped across all Tenants, and Tenant-level roles scoped to a single Tenant's data, with granular module-level RBAC permissions within each Tenant.

- Tenant Admins can create custom roles and assign module/action-level permissions within their own Tenant.
- Supports multi-branch/multi-location access restriction per user within a Tenant.

### Database Entities

| Entity             | Key Attributes                                                                         | Notes                                            |
|--------------------|----------------------------------------------------------------------------------------|--------------------------------------------------|
| user               | id, tenant_id (nullable for platform roles), name, email, phone, password_hash, status | tenant_id is null only for Platform Super Admin. |
| role               | id, tenant_id (nullable for system roles), name, is_custom                             | Predefined + Tenant-custom roles.                |
| permission         | id, module, action<br>(view/create/edit/delete/approve)                                | Granular permission catalog.                     |
| role_permission    | role_id, permission_id                                                                 | Many-to-many mapping.                            |
| user_branch_access | user_id, branch_id                                                                     | Multi-branch restriction.                        |

### APIs Required

| Method & Path             | Purpose                                                                |
|---------------------------|------------------------------------------------------------------------|
| POST /api/v1/auth/login   | Authenticate, issue JWT access/refresh tokens (sample in Section 7.5). |
| POST /api/v1/auth/refresh | Rotate access token using refresh token.                               |
| POST /api/v1/users        | Create a Tenant user (Tenant Admin).                                   |

| Method & Path                  | Purpose                                         |
|--------------------------------|-------------------------------------------------|
| GET /api/v1/roles              | List system and custom roles for the Tenant.    |
| POST /api/v1/roles             | Create a custom role with selected permissions. |
| PATCH /api/v1/users/{id}/roles | Assign/change a user's role(s).                 |

### User Roles

- Platform Super Admin – manages platform-role accounts.
- Tenant Admin – manages all users/roles within own Tenant.

### Integration Requirements

- Optional OTP/2FA provider (SMS/email) for login hardening (Section 8.1).

## 5.3 Lead Management

### Functional Description

Captures and qualifies prospective CHS, Commercial, and Single Shop customers prior to conversion, per SRS Section 4.2. Included as a foundation module because Sales Order Management and Quotation Management (Sections 5.11–5.12) both originate from a converted lead.

- Pipeline stages: New → Contacted → Site Survey Scheduled → Quoted → Negotiation → Converted/Lost.
- One-click conversion of a qualified lead into a Customer Master record and initial Quotation without re-entry.

### Database Entities

| Entity        | Key Attributes                                                    | Notes                             |
|---------------|-------------------------------------------------------------------|-----------------------------------|
| lead          | id, tenant_id, category, source, status, assigned_to, lost_reason | Pre-customer record.              |
| lead_activity | id, lead_id, type, notes, next_action_at                          | Call logs, follow-ups, reminders. |

### APIs Required

| Method & Path                              | Purpose                                                         |
|--------------------------------------------|-----------------------------------------------------------------|
| POST /api/v1/leads                         | Capture a new lead.                                             |
| GET /api/v1/leads?status=&category=&owner= | List/filter the lead pipeline.                                  |
| PATCH /api/v1/leads/{id}/status            | Move a lead through the pipeline; capture lost reason.          |
| POST /api/v1/leads/{id}/convert            | Convert lead → Customer Master + draft Quotation in one action. |
| POST /api/v1/leads/{id}/activities         | Log a call/follow-up/reminder.                                  |

### User Roles

- Sales Executive – owns assigned leads end-to-end.
- Tenant Admin / Operations Manager – pipeline visibility and reassignment.
- Auditor – read-only.

### Integration Requirements

- SMS/Email/WhatsApp for follow-up reminders (Section 9).
- Website/contact-form webhook as an additional lead source (future-ready endpoint).

## 5.4 Customer & Society Management

### Functional Description

Maintains the unified Customer Master spanning CHS (Cooperative Housing Society), Commercial, and Single Shop categories (SRS Section 4.3), each with multiple contacts, sites, and a full interaction history.

- CHS profile: society name, RWA/managing-committee contacts, tower/wing/unit counts, registration number.
- Commercial profile: branch locations, billing/shipping addresses, GSTIN, authorized signatory.
- Single Shop profile: simplified billing, GSTIN optional for unregistered businesses.

### Database Entities

| Entity           | Key Attributes                               | Notes                                                         |
|------------------|----------------------------------------------|---------------------------------------------------------------|
| customer         | id, tenant_id, category, name, status, gstin | CHS / Commercial / Single Shop.                               |
| customer_contact | id, customer_id, name, role, phone, email    | Multiple contacts per customer.                               |
| site             | id, customer_id, address, geo_lat, geo_lng   | One or more physical locations (links to Section 5.6 assets). |

### APIs Required

| Method & Path                              | Purpose                                                   |
|--------------------------------------------|-----------------------------------------------------------|
| POST /api/v1/customers                     | Create a customer (direct entry or from lead conversion). |
| GET /api/v1/customers?category=&status=&q= | Search/filter Customer Master.                            |
| GET /api/v1/customers/{id}                 | Full profile incl. sites, contacts, interaction history.  |
| POST /api/v1/customers/{id}/contacts       | Add a contact person.                                     |
| POST /api/v1/customers/{id}/sites          | Add a physical site.                                      |
| GET /api/v1/customers/{id}/timeline        | Calls, leads, complaints, visits, invoices in one feed.   |

**User Roles**

- Sales Executive – create/update.
- Operations Manager, Service Coordinator, Accounts/Billing – read, module-scoped update.
- Society/Client Admin – self-service read of own record via Customer Portal.
- Auditor – read-only.

**Integration Requirements**

- Document store for society registration documents / authorized-signatory proofs.

## 5.5 Vendor Management

**Functional Description**

Maintains the unified Vendor Master for equipment suppliers, spare-parts vendors, and outsourced service partners (SRS Section 4.4), feeding Inventory replenishment (Section 5.10) and vendor payables.

**Database Entities**

| Entity         | Key Attributes                                              | Notes                            |
|----------------|-------------------------------------------------------------|----------------------------------|
| vendor         | id, tenant_id, name, category, gstin, status, payment_terms | Active / Inactive / Blacklisted. |
| vendor_contact | id, vendor_id, name, phone, email                           | Multiple contacts.               |
| purchase_order | id, tenant_id, vendor_id, status, total_amount              | Procurement document.            |
| vendor_payable | id, vendor_id, invoice_ref, amount, status                  | Outstanding payable balances.    |

**APIs Required**

| Method & Path                             | Purpose                                                            |
|-------------------------------------------|--------------------------------------------------------------------|
| POST /api/v1/vendors                      | Create a vendor.                                                   |
| GET /api/v1/vendors?status=&category=     | List/filter vendors.                                               |
| POST /api/v1/vendors/{id}/purchase-orders | Raise a purchase order linked to a reorder request (Section 5.10). |
| GET /api/v1/vendors/{id}/payables         | Outstanding payable balance and history.                           |
| POST /api/v1/vendors/{id}/payments        | Record a payment made to the vendor.                               |

**User Roles**

- Procurement/Vendor Manager – full access.
- Operations Manager, Accounts/Billing – read.
- Auditor – read-only.

### Integration Requirements

- Email for purchase-order issuance to vendors.

## 5.6 AMC Management

### Functional Description

Manages the AMC contract lifecycle linked to one or more customer sites (SRS Section 4.7): terms, coverage, billing cycle, status, renewal reminders, and full contract history.

- Contract status: Draft → Active → Expiring Soon → Expired → Renewed → Cancelled.
- Auto-triggers preventive-maintenance scheduling (Section 5.8) based on contract terms.
- Auto-creation on installation handover (SRS Section 4.5).

### Database Entities

| Entity               | Key Attributes                                                                                                   | Notes                       |
|----------------------|------------------------------------------------------------------------------------------------------------------|-----------------------------|
| amc_contract         | id, tenant_id, customer_id, site_id, start_date, end_date, coverage_type, visits_included, billing_cycle, status | Core contract record.       |
| amc_contract_version | id, amc_contract_id, version_no, snapshot_json                                                                   | Full version/history trail. |

### APIs Required

| Method & Path                                      | Purpose                                                               |
|----------------------------------------------------|-----------------------------------------------------------------------|
| POST /api/v1/amc-contracts                         | Create a contract (from approved quotation or installation handover). |
| GET /api/v1/amc-contracts?status=&expiring_within= | List/filter, including expiring-soon view.                            |
| POST /api/v1/amc-contracts/{id}/renew              | Renew, generating a new version and updated schedule.                 |
| GET /api/v1/amc-contracts/{id}/history             | Full version/amendment history.                                       |
| POST /api/v1/amc-contracts/{id}/cancel             | Cancel with reason code.                                              |

### User Roles

- Operations Manager – owns contract lifecycle.
- Sales Executive – initiates from converted lead/quotation.
- Society/Client Admin – read own contract status via portal.
- Auditor – read-only.

### Integration Requirements

- Notification Engine for renewal reminders (Section 9).
- Invoice Management for recurring AMC billing (Section 5.13).

## 5.7 Service Ticket Management

### Functional Description

Logs and tracks service requests/complaints against a site or asset through to resolution, with SLA timers and escalation (SRS Section 4.8), and underpins both the customer portal complaint flow and technician dispatch.

### Database Entities

| Entity            | Key Attributes                                                           | Notes                                                         |
|-------------------|--------------------------------------------------------------------------|---------------------------------------------------------------|
| service_ticket    | id, tenant_id, site_id, asset_id, category, priority, status, sla_due_at | Open → Assigned → In Progress → Resolved → Closed → Reopened. |
| ticket_comment    | id, ticket_id, user_id, comment, attachment_url                          | Audit trail of comments/attachments.                          |
| ticket_status_log | id, ticket_id, from_status, to_status, changed_by, changed_at            | Full status-change audit trail.                               |

### APIs Required

| Method & Path                                       | Purpose                                                        |
|-----------------------------------------------------|----------------------------------------------------------------|
| POST /api/v1/tickets                                | Raise a ticket (customer portal, call-center staff, or admin). |
| GET /api/v1/tickets?status=&priority=&sla_breached= | Ticket board / queue view.                                     |
| PATCH /api/v1/tickets/{id}/assign                   | Assign to a technician (Section 5.8).                          |
| PATCH /api/v1/tickets/{id}/status                   | Transition status; auto-logs to ticket_status_log.             |
| POST /api/v1/tickets/{id}/comments                  | Add comment/attachment.                                        |

### User Roles

- Service Coordinator – assigns and tracks tickets.
- Field Technician – updates assigned tickets via mobile.
- Society/Client Admin – raises/tracks own tickets via portal.
- Auditor – read-only.

### Integration Requirements

- Notification Engine for SLA breach escalation (Section 9).
- Engineer Visit Management for dispatch and field execution.

## 5.8 Engineer Visit Management (incl. Preventive Maintenance)

### Functional Description

Covers technician assignment, dispatch, and field execution for both reactive (ticket-driven) and scheduled (AMC preventive-maintenance) visits (SRS Sections 4.9–4.10), including GPS check-in/out and productivity tracking.



- Preventive visits are auto-generated from AMC contract terms; Service Coordinators can manually reschedule/skip with reason codes.
- Technicians log work performed, parts consumed, before/after photos, and signature/OTP confirmation (Section 5.9).

### Database Entities

| Entity             | Key Attributes                                                                                                                  | Notes                                |
|--------------------|---------------------------------------------------------------------------------------------------------------------------------|--------------------------------------|
| engineer_visit     | id, tenant_id, ticket_id (nullable), amc_contract_id (nullable), technician_id, scheduled_at, checkin_gps, checkout_gps, status | Unifies ticket visits and PM visits. |
| visit_parts_used   | id, visit_id, inventory_item_id, qty                                                                                            | Drives inventory auto-deduction.     |
| technician_profile | id, user_id, skills, territory, availability                                                                                    | Skill/location-based assignment.     |

### APIs Required

| Method & Path                             | Purpose                                                     |
|-------------------------------------------|-------------------------------------------------------------|
| GET /api/v1/visits?technician_id=&date=   | Technician's schedule / dispatch board.                     |
| POST /api/v1/visits                       | Schedule a visit (manual or system-generated from AMC).     |
| PATCH /api/v1/visits/{id}/checkin         | Log GPS + timestamp on arrival.                             |
| PATCH /api/v1/visits/{id}/complete        | Log work done, parts used, photos, checkout GPS, signature. |
| GET /api/v1/technicians/{id}/productivity | Visit count, average resolution time.                       |

### User Roles

- Service Coordinator – schedules/reassigns visits.
- Field Technician – executes visits via mobile app.
- Society/Client Admin – views scheduled/completed visits.
- Auditor – read-only.

### Integration Requirements

- Mobile App Module (Section 5.18) for field execution.
- Inventory Management for parts auto-deduction.
- Notification Engine for visit reminders.

## 5.9 Digital Signature Capture

### Functional Description

Captures customer sign-off / OTP confirmation at installation handover and at completion of each service visit (SRS Sections 4.5 and 4.10), producing a legally-referenceable record attached to the relevant job.

**Database Entities**

| Entity            | Key Attributes                                                                                 | Notes                                             |
|-------------------|------------------------------------------------------------------------------------------------|---------------------------------------------------|
| signature_capture | id, tenant_id, ref_type (visit/installation), ref_id, signature_url, captured_at, otp_verified | Polymorphic link to visit or installation record. |

**APIs Required**

| Method & Path                              | Purpose                                                                                       |
|--------------------------------------------|-----------------------------------------------------------------------------------------------|
| POST /api/v1/signatures                    | Upload a captured signature image (multipart) or confirm OTP, linked to a visit/installation. |
| GET /api/v1/signatures/{ref_type}/{ref_id} | Retrieve the signature/OTP record for a job.                                                  |

**User Roles**

- Field Technician – captures via mobile app.
- Society/Client Admin – provides signature/OTP; views own history.

**Integration Requirements**

- Object storage for signature image files.
- SMS/WhatsApp gateway for OTP delivery and verification.
- Engineer Visit Management and New Installation flows as the consuming modules.

## 5.10 Inventory Management

**Functional Description**

Maintains spare parts, cameras, cables, and accessories with stock levels across warehouses and technician van-stock, per Tenant, with auto-deduction on consumption and vendor-linked reorder workflows (SRS Section 4.11).

**Database Entities**

| Entity          | Key Attributes                                                                               | Notes                                       |
|-----------------|----------------------------------------------------------------------------------------------|---------------------------------------------|
| inventory_item  | id, tenant_id, sku, name, category, reorder_level                                            | Master item catalog.                        |
| warehouse       | id, tenant_id, name, type (central/branch/van)                                               | Stock locations incl. technician van-stock. |
| stock_ledger    | id, item_id, warehouse_id, txn_type (receipt/issue/return/adjustment), qty, ref_type, ref_id | Full movement traceability.                 |
| reorder_request | id, item_id, vendor_id, qty, status                                                          | Links to Vendor Management purchase orders. |

**APIs Required**

| Method & Path                              | Purpose                                                                |
|--------------------------------------------|------------------------------------------------------------------------|
| GET /api/v1/inventory/items?low_stock=true | Catalog with low-stock alerting.                                       |
| POST /api/v1/inventory/stock-movements     | Record receipt/issue/return/adjustment.                                |
| POST /api/v1/inventory/reorder-requests    | Raise a reorder, optionally auto-from low-stock alert.                 |
| GET /api/v1/inventory/items/{id}/ledger    | Full movement history for an item.                                     |
| POST /api/v1/inventory/consume             | Internal API called by Engineer Visit completion to auto-deduct parts. |

**User Roles**

- Procurement/Vendor Manager – reorder workflow.
- Service Coordinator, Field Technician – issue/consume stock.
- Operations Manager – stock health oversight.
- Auditor – read-only.

**Integration Requirements**

- Vendor Management for reorder/purchase orders.
- Engineer Visit Management for consumption-driven deduction.

**5.11 Quotation Management****Functional Description**

Generates itemized quotations for new installations, AMC plans, upgrades, or repairs across all customer categories, with digital approval and direct conversion into AMC contracts or sales/installation orders (SRS Section 4.12).

**Database Entities**

| Entity              | Key Attributes                                                                                       | Notes                                                |
|---------------------|------------------------------------------------------------------------------------------------------|------------------------------------------------------|
| quotation           | id, tenant_id, customer_id, lead_id (nullable), status, subtotal, tax_amount, discount, total_amount | Draft → Sent → Viewed → Approved/Rejected → Expired. |
| quotation_line_item | id, quotation_id, description, qty, unit_price, tax_pct                                              | Equipment, labor, taxes, discounts.                  |

**APIs Required**

| Method & Path           | Purpose                                         |
|-------------------------|-------------------------------------------------|
| POST /api/v1/quotations | Create a quotation (standalone or from a lead). |
| GET /api/v1/quotations? | List/filter quotations.                         |

| Method & Path                        | Purpose                                                      |
|--------------------------------------|--------------------------------------------------------------|
| status=&customer_id=                 |                                                              |
| POST /api/v1/quotations/{id}/send    | Mark sent; share via email/WhatsApp; generate PDF.           |
| POST /api/v1/quotations/{id}/respond | Customer portal: approve or reject digitally.                |
| POST /api/v1/quotations/{id}/convert | Convert approved quotation into AMC contract or Sales Order. |

### User Roles

- Sales Executive – creates/sends quotations.
- Operations Manager – approval oversight.
- Society/Client Admin – views and approves/rejects via portal.
- Auditor – read-only.

### Integration Requirements

- Document/PDF Generation Service for quotation PDFs.
- Notification Engine for send/viewed/expiry alerts.
- Sales Order and AMC Management as downstream conversions.

## 5.12 Sales Order Management

### Functional Description

Represents the work order created from an approved Quotation for new installations, upgrades, or one-off paid work (operationalizing SRS Section 4.5's installation work order, generalized to any billable order type), tracked through to invoicing.

### Database Entities

| Entity            | Key Attributes                                                                                                | Notes                                                                        |
|-------------------|---------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------|
| sales_order       | id, tenant_id, quotation_id, customer_id, type (installation/upgrade/repair), status, assigned_technician_ids | Survey Pending → Material Allocated → In Progress → Completed → Handed Over. |
| sales_order_asset | id, sales_order_id, asset_id                                                                                  | Assets installed/handled under this order, links to Site & Asset register.   |

### APIs Required

| Method & Path                           | Purpose                                                                   |
|-----------------------------------------|---------------------------------------------------------------------------|
| POST /api/v1/sales-orders               | Create from an approved quotation.                                        |
| GET /api/v1/sales-orders?status=        | Track installation/work-order pipeline.                                   |
| PATCH /api/v1/sales-orders/{id}/status  | Update job status through to Handed Over.                                 |
| POST /api/v1/sales-orders/{id}/handover | Capture handover checklist, photos, signature; auto-trigger AMC creation. |

### User Roles

- Operations Manager – owns work-order execution.
- Service Coordinator – assigns technicians.
- Field Technician – updates progress via mobile.
- Auditor – read-only.

### Integration Requirements

- Site & Asset register for installed-asset capture.
- Digital Signature Capture for handover sign-off.
- AMC Management auto-triggered on handover.
- Invoice Management for billing the completed order.

## 5.13 Invoice Management

### Functional Description

Generates GST-compliant tax invoices for installations, AMC billing (including recurring cycles), and repair work, with simplified non-GST billing for unregistered single-shop clients, plus credit notes and revision history (SRS Section 4.13).

### Database Entities

| Entity            | Key Attributes                                                                                                                  | Notes                                                         |
|-------------------|---------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------|
| invoice           | id, tenant_id, customer_id, sales_order_id (nullable), amc_contract_id (nullable), invoice_no, gst_amount, total_amount, status | Draft → Issued → Partially Paid → Paid → Overdue → Cancelled. |
| invoice_line_item | id, invoice_id, description, qty, unit_price, tax_pct                                                                           | Itemized billing lines.                                       |
| credit_note       | id, invoice_id, amount, reason                                                                                                  | Adjustments/revisions.                                        |

### APIs Required

| Method & Path                             | Purpose                                                      |
|-------------------------------------------|--------------------------------------------------------------|
| POST /api/v1/invoices                     | Generate an invoice from a sales order or AMC billing cycle. |
| GET /api/v1/invoices?status=&customer_id= | List/filter invoices.                                        |
| POST /api/v1/invoices/{id}/credit-notes   | Issue a credit note against an invoice.                      |
| GET /api/v1/invoices/{id}/pdf             | Download invoice PDF.                                        |
| POST /api/v1/invoices/{id}/send           | Share via email/WhatsApp.                                    |

### User Roles

- Accounts/Billing Staff – full lifecycle.

- Operations Manager – read/oversight.
- Society/Client Admin – downloads own invoices via portal.
- Auditor – read-only.

### Integration Requirements

- Document/PDF Generation Service.
- Notification Engine for invoice delivery and payment-due reminders.
- Payment Tracking as the downstream module.

## 5.14 Payment Tracking

### Functional Description

Records customer payments against invoices across all modes, tracks ageing and outstanding dues with automated reminders, generates receipts, and tracks vendor payable balances (SRS Section 4.14).

### Database Entities

| Entity          | Key Attributes                                                                                 | Notes                         |
|-----------------|------------------------------------------------------------------------------------------------|-------------------------------|
| payment         | id, tenant_id, invoice_id, mode (cash/cheque/bank/UPI/card), amount, received_at, reference_no | Customer receipts.            |
| receipt         | id, payment_id, receipt_no, pdf_url                                                            | Generated receipt document.   |
| customer_ledger | id, customer_id, balance, last_updated                                                         | Running balance per customer. |

### APIs Required

| Method & Path                                    | Purpose                                              |
|--------------------------------------------------|------------------------------------------------------|
| POST /api/v1/payments                            | Record a payment against an invoice.                 |
| GET /api/v1/payments?customer_id=&ageing_bucket= | Outstanding dues with ageing (0-30/31-60/61-90/90+). |
| GET /api/v1/payments/{id}/receipt                | Download generated receipt.                          |
| GET /api/v1/customers/{id}/ledger                | Running payment ledger for a customer.               |

### User Roles

- Accounts/Billing Staff – full access.
- Operations Manager – oversight/reporting.
- Society/Client Admin – views own ledger/receipts.
- Auditor – read-only.

### Integration Requirements

- Integration-ready hooks for online payment gateways (Phase 2 per SRS Section 7).

- Notification Engine for payment-reminder dispatch.

## 5.15 Notification Engine

### Functional Description

Single, centralized engine that every other module calls to send Email, SMS, WhatsApp, or in-app notifications, with configurable templates per event type and recipient role (SRS Section 4.15). Full design in Section 9.

### Database Entities

| Entity                | Key Attributes                                                 | Notes                               |
|-----------------------|----------------------------------------------------------------|-------------------------------------|
| notification_template | id, tenant_id, event_type, channel, template_body              | Configurable per Tenant/event.      |
| notification_log      | id, tenant_id, event_type, channel, recipient, status, sent_at | Delivery tracking.                  |
| in_app_notification   | id, user_id, title, body, read_at                              | Internal staff notification center. |

### APIs Required

| Method & Path                                | Purpose                                                                                  |
|----------------------------------------------|------------------------------------------------------------------------------------------|
| POST /api/v1/notifications/send              | Internal API: enqueue a notification (called by other modules' services, not end users). |
| GET /api/v1/notifications/templates          | Manage Tenant-level templates.                                                           |
| GET /api/v1/notifications/in-app             | Fetch current user's in-app notification feed.                                           |
| PATCH /api/v1/notifications/in-app/{id}/read | Mark as read.                                                                            |

### User Roles

- Tenant Admin – configures templates.
- All internal roles – receive in-app notifications relevant to their role.

### Integration Requirements

- Email gateway (SMTP/API), SMS gateway (DLT-registered), WhatsApp Business API.
- Triggered by AMC, Ticket, Payment, Lead, and Visit modules (Section 9.2).

## 5.16 Reporting & Analytics

### Functional Description

Provides standard operational and financial reports with export and date/filter-based generation (SRS Section 4.16). Full design, KPI framework, and aggregation strategy in Section 11.

**Database Entities**

| Entity            | Key Attributes                                    | Notes                         |
|-------------------|---------------------------------------------------|-------------------------------|
| report_definition | id, tenant_id, name, query_ref, parameters_schema | Catalog of available reports. |
| report_export_log | id, report_id, requested_by, format, generated_at | Export audit trail.           |

**APIs Required**

| Method & Path                                   | Purpose                                             |
|-------------------------------------------------|-----------------------------------------------------|
| GET /api/v1/reports                             | List available reports for the user's role.         |
| POST /api/v1/reports/{id}/run                   | Execute a report with date-range/filter parameters. |
| GET /api/v1/reports/{id}/export?format=xlsx pdf | Export to Excel/PDF (sample in Section 7.5).        |

**User Roles**

- All Tenant roles – scoped to reports relevant to their role.
- Auditor – compliance and historical reports.
- Platform Super Admin – platform-wide usage reports.

**Integration Requirements**

- Read replica for report query offload (Section 6.7).
- Export pipeline to Excel/PDF generation service.

**5.17 Dashboard Module****Functional Description**

Role-based landing dashboards summarizing lead pipeline, open tickets, SLA compliance, revenue, and AMC renewal pipeline (SRS Section 4.16). Architecture detailed in Section 11.1.

**Database Entities**

| Entity                  | Key Attributes                             | Notes                                   |
|-------------------------|--------------------------------------------|-----------------------------------------|
| dashboard_widget_config | id, tenant_id, role, widget_key, position  | Per-role widget layout.                 |
| kpi_snapshot            | id, tenant_id, kpi_key, value, computed_at | Pre-aggregated KPI cache for fast load. |

**APIs Required**

| Method & Path                 | Purpose                                              |
|-------------------------------|------------------------------------------------------|
| GET /api/v1/dashboard/summary | Role-appropriate KPI summary for the logged-in user. |



| Method & Path                       | Purpose                                |
|-------------------------------------|----------------------------------------|
| GET /api/v1/dashboard/widgets/{key} | Drill-down data for a specific widget. |

### User Roles

- All Tenant roles – see a role-appropriate dashboard.
- Platform Super Admin – platform health dashboard.

### Integration Requirements

- Background worker pre-computes kpi\_snapshot rows on a schedule to keep dashboard load under the 3-second target (SRS Section 5.1).

## 5.18 Mobile App Module (Field Technician App)

### Functional Description

React Native app giving field technicians their assigned tickets, installation jobs, and visits, with offline capture and sync, GPS, photos, signatures, and parts-used logging (SRS Section 4.18). Full architecture in Section 10.

### Database Entities

| Entity                    | Key Attributes                                                               | Notes                                                                          |
|---------------------------|------------------------------------------------------------------------------|--------------------------------------------------------------------------------|
| (reuses backend entities) | engineer_visit, service_ticket, sales_order, signature_capture, stock_ledger | No separate mobile-only schema; mobile reads/writes the same entities via API. |
| device_session            | id, user_id, device_id, push_token, last_sync_at                             | Push registration and sync bookkeeping.                                        |

### APIs Required

| Method & Path                        | Purpose                                                          |
|--------------------------------------|------------------------------------------------------------------|
| GET /api/v1/mobile/visits/today      | Technician's day-schedule, optimized payload for mobile.         |
| POST /api/v1/mobile/sync             | Batched upload of offline-queued changes; returns server deltas. |
| POST /api/v1/mobile/devices/register | Register device/push token (Section 10.6).                       |

### User Roles

- Field Technician – primary user.
- Service Coordinator – visibility into technician status via web (not the app itself).

### Integration Requirements

- FCM/APNs for push notifications.
- Object storage for offline-captured media upload on reconnect.

## 6. Database Architecture

### 6.1 Database Design Principles

- **Tenant-scoped by default:** every business table carries a non-nullable `tenant_id` (foreign key to `tenant.id`); the only exceptions are genuinely platform-global tables (`tenant`, `subscription_plan`, `permission catalog`).
- **Single shared schema:** one PostgreSQL schema serves all Tenants (Section 2.4); isolation is enforced through application-layer filtering plus PostgreSQL Row-Level Security, not through per-Tenant schemas or databases.
- **Normalized core, denormalized read paths:** the transactional schema (Section 6.3–6.4) is normalized to 3NF for integrity; reporting/dashboard reads are served from pre-aggregated `kpi_snapshot`-style tables or the read replica, not by denormalizing the transactional tables themselves.
- **Auditability by construction:** every table that represents a business document (quotation, invoice, AMC contract, ticket) has a status-history or version table alongside it, and all writes additionally land in the central `audit_log` (Section 8.6).
- **Soft delete over hard delete:** business records are deactivated (`status/is_active`) rather than physically deleted, preserving referential and audit integrity; physical deletion is reserved for GDPR/data-retention-driven erasure workflows.
- **UUID primary keys for Tenant-facing entities:** avoids leaking sequential row counts across Tenants and simplifies future data export/migration per Tenant.

### 6.2 Entity-Relationship Diagram

Figure 6 shows the core entities and relationships spanning the sales, operations, and finance lifecycle. Every entity shown carries a `tenant_id` (omitted from the diagram for readability) except where it is itself a child of another Tenant-scoped entity.

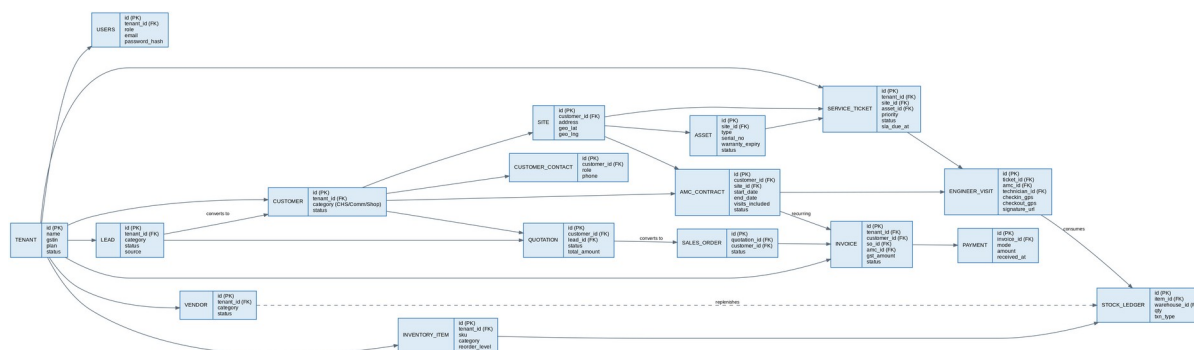


Figure 6: Database Entity-Relationship Diagram (Core Entities)

### 6.3 Master Tables

| Table                  | Purpose                                                                       |
|------------------------|-------------------------------------------------------------------------------|
| tenant                 | Root record for each onboarded CCTV AMC service-provider company.             |
| subscription_plan      | Platform-defined plans (Starter/Growth/Enterprise) with feature/usage limits. |
| user, role, permission | Identity, RBAC roles, and the module/action permission catalog.               |

| Table                      | Purpose                                                            |
|----------------------------|--------------------------------------------------------------------|
| customer, customer_contact | Unified Customer Master (CHS/Commercial/Single Shop) and contacts. |
| site, asset                | Physical locations and CCTV/NVR/DVR asset register.                |
| vendor, vendor_contact     | Supplier/service-partner master.                                   |
| inventory_item, warehouse  | Spare-parts/equipment catalog and stock locations.                 |
| notification_template      | Per-Tenant, per-event notification content.                        |
| report_definition          | Catalog of available reports.                                      |

## 6.4 Transaction Tables

| Table                                               | Purpose                                                             |
|-----------------------------------------------------|---------------------------------------------------------------------|
| lead, lead_activity                                 | Sales pipeline records and follow-up activity.                      |
| quotation, quotation_line_item                      | Itemized quotations and approval workflow.                          |
| sales_order, sales_order_asset                      | Installation/upgrade/repair work orders.                            |
| amc_contract, amc_contract_version                  | AMC contracts and their version history.                            |
| service_ticket, ticket_comment, ticket_status_log   | Complaints/service requests and their audit trail.                  |
| engineer_visit, visit_parts_used, signature_capture | Field execution records, parts consumed, sign-off.                  |
| stock_ledger, reorder_request, purchase_order       | Inventory movement and procurement.                                 |
| invoice, invoice_line_item, credit_note             | Billing documents.                                                  |
| payment, receipt, customer_ledger                   | Collections and running balances.                                   |
| notification_log, in_app_notification               | Notification delivery and in-app feed.                              |
| audit_log                                           | Tamper-evident record of all create/update/delete/approval actions. |

## 6.5 Relationship Mapping

Primary cardinalities (see Figure 6 for the full graph):

- tenant 1—N user, customer, vendor, inventory\_item, service\_ticket, invoice (every Tenant-scoped table).

- lead 1—1 customer (on conversion) and lead 1—N quotation.
- customer 1—N site, customer 1—N customer\_contact, customer 1—N amc\_contract.
- site 1—N asset, site 1—N service\_ticket.
- amc\_contract 1—N engineer\_visit (preventive) and 1—N invoice (recurring billing).
- service\_ticket 1—N engineer\_visit (reactive) and 1—N ticket\_comment / ticket\_status\_log.
- quotation 1—1 sales\_order (on conversion); sales\_order 1—N invoice.
- invoice 1—N payment and 1—N credit\_note; payment 1—1 receipt.
- inventory\_item 1—N stock\_ledger; vendor 1—N purchase\_order / reorder\_request.

## 6.6 Indexing Strategy

| Pattern                                                | Index Type       | Why                                                                                 |
|--------------------------------------------------------|------------------|-------------------------------------------------------------------------------------|
| (tenant_id, status)                                    | Composite B-tree | Every "my Tenant's open/active records" query — tickets, AMC contracts, quotations. |
| (tenant_id, created_at DESC)                           | Composite B-tree | Recent-first list views and pagination.                                             |
| (customer_id)                                          | B-tree (FK)      | Customer 360° timeline assembly across sites, tickets, invoices.                    |
| (tenant_id, sla_due_at) WHERE status NOT IN ('Closed') | Partial index    | SLA-breach monitoring scans only open tickets.                                      |
| (tenant_id, end_date) WHERE status = 'Active'          | Partial index    | AMC expiring-soon and warranty-expiry alert jobs.                                   |
| GIN on to_tsvector(name/address)                       | Full-text        | Customer/site/asset search.                                                         |
| (item_id, warehouse_id, created_at)                    | Composite B-tree | Stock ledger running-balance computation.                                           |
| (tenant_id, event_type, sent_at)                       | Composite B-tree | Notification log lookups and dedup checks.                                          |

## 6.7 Database Optimization Strategy

- **Read/write split:** all reporting, dashboard, and export queries (Sections 5.16–5.17, 11) run against a PostgreSQL read replica; the primary is reserved for transactional reads/writes.
- **Connection pooling:** PgBouncer (or equivalent) in front of PostgreSQL keeps connection counts bounded as the FastAPI tier scales horizontally (Section 13).
- **Partitioning for high-growth tables:** audit\_log, notification\_log, and stock\_ledger are partitioned by month (and, where Tenant volume warrants it, additionally by tenant\_id range) to keep indexes small and vacuum/maintenance fast.
- **Caching layer:** Redis caches Tenant branding/config, role-permission matrices, and computed kpi\_snapshot values, with explicit invalidation on the relevant write path.
- **Query governance:** all repository methods are reviewed for index coverage in CI via automated EXPLAIN checks on critical queries; N+1 query patterns are caught via SQLAlchemy eager-loading conventions documented in Section 15.

- **Row-Level Security:** RLS policies on every Tenant-scoped table provide a database-enforced backstop to the application-layer `tenant_id` filter (Section 2.4), at negligible query-plan cost when paired with the `(tenant_id, ...)` composite indexes above.
- **Backups & retention:** automated daily full backups plus continuous WAL archiving for point-in-time recovery; backup/DR procedure detailed in Section 8.9.

## 7. API Architecture

---

### 7.1 API Standards

- REST over HTTPS only; all payloads are JSON (application/json).
- Resource-oriented, plural-noun paths: /api/v1/customers, /api/v1/tickets/{id}, /api/v1/tickets/{id}/comments.
- snake\_case for JSON field names and query parameters; ISO 8601 for all dates/timestamps, always in UTC on the wire (clients render in local time).
- Every endpoint is documented automatically via FastAPI's generated OpenAPI schema — no hand-maintained API reference is kept separately.
- Idempotency: all state-changing POSTs that create a business document accept an optional Idempotency-Key header to safely tolerate client retries (important on the mobile app's flaky-network paths).

### 7.2 Authentication Flow

All authentication is JWT-based. Figure 7 shows the full request path from client to database, including where authentication, tenant resolution, and authorization are enforced.

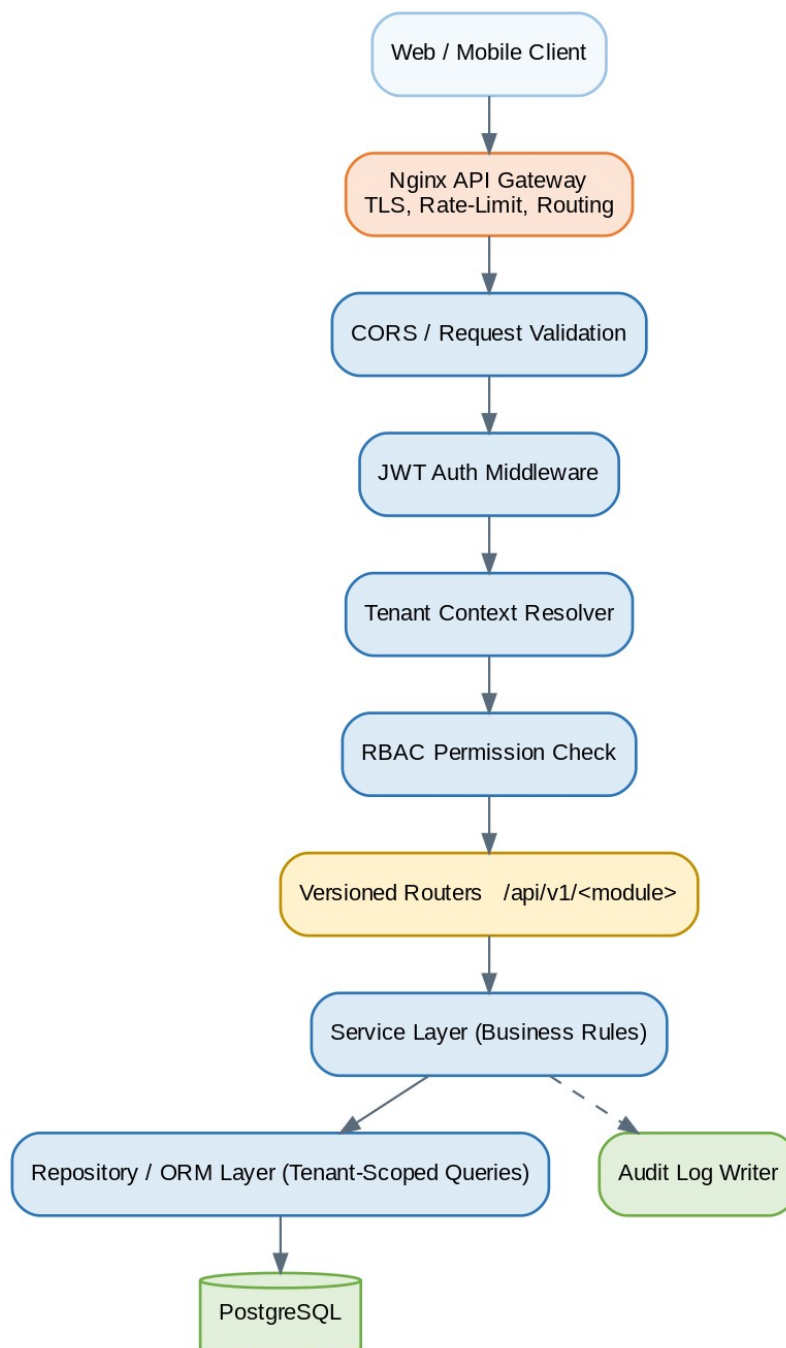


Figure 7: API Request Flow – Gateway, Middleware Chain, and Layering

1. Client calls POST /api/v1/auth/login with email + password (+ OTP if 2FA is enabled for the Tenant).
2. API verifies the password hash (argon2/bcrypt) and, on success, issues a short-lived JWT access token (~15 minutes) and a longer-lived, rotating refresh token (~7–30 days), both signed server-side.
3. The access token's claims include user\_id, tenant\_id, role, and an issued-at/expiry; the client sends it as Authorization: Bearer <token> on every subsequent request.

4. The JWT Auth Middleware verifies the token signature and expiry on every request; the Tenant Context Resolver reads `tenant_id` from the verified claims (never from a client-supplied field) and binds it to the request-scoped DB session.
5. When the access token expires, the client calls `POST /api/v1/auth/refresh` with the refresh token; the server validates it against its stored (hashed) value, rotates it, and issues a new access token.

## 7.3 Authorization Model

Authorization is RBAC, evaluated after authentication and tenant resolution (Section 2.4), with two tiers per the SRS (Section 4.20):

| Tier                 | Scope              | Example                                                                                                                                                                |
|----------------------|--------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Platform-level roles | Across all Tenants | Platform Super Admin managing Tenant onboarding/subscriptions.                                                                                                         |
| Tenant-level roles   | Single Tenant only | Tenant Admin, Sales Executive, Operations Manager, Service Coordinator, Field Technician, Accounts/Billing, Procurement/Vendor Manager, Society/Client Admin, Auditor. |

Each Tenant-level role maps to a permission set of (module, action) pairs — action  $\in$  {view, create, edit, delete, approve}. Tenant Admins can clone a system role into a custom role and adjust its permission set without code changes (SRS Section 4.20). Every route declares its required (module, action) via a FastAPI dependency, so permission checks cannot be accidentally skipped per-route.

## 7.4 API Gateway Design

- **Edge (Nginx):** TLS termination, gzip, static asset caching, coarse rate-limiting (per-IP), and load-balancing across FastAPI container replicas.
- **Application gateway (FastAPI middleware chain):** CORS validation → JWT authentication → tenant context resolution → RBAC permission check → versioned router dispatch (Figure 7).
- **Per-Tenant rate limiting:** applied at the application layer (keyed by `tenant_id`, backed by Redis counters) so one Tenant's traffic spike cannot degrade another Tenant's experience — directly supporting the SRS's per-Tenant resource-limit requirement (Section 5.3).

## 7.5 Request / Response Standards

All responses use a consistent envelope so frontend and mobile clients can handle success/error/pagination uniformly:

```
// Success (single resource)
{
  "success": true,
  "data": { "id": "8f1e...", "name": "Sunrise CHS", "category": "CHS" },
  "meta": null
}

// Success (paginated list)
{
  "success": true,
  "data": [ { "id": "...", "...": "..." } ],
  "meta": { "page": 1, "page_size": 20, "total": 134 }
```



```
}
```

## 7.6 Error Handling Standards

```
// Error envelope
{
  "success": false,
  "error": {
    "code": "TICKET_SLA_ALREADY_BREACHED",
    "message": "Cannot reassign a ticket after SLA breach without an override.",
    "details": { "ticket_id": "T-2026-00451" }
  }
}
```

| HTTP Status               | Usage                                                                                              |
|---------------------------|----------------------------------------------------------------------------------------------------|
| 400 Bad Request           | Payload fails Pydantic schema validation.                                                          |
| 401 Unauthorized          | Missing/invalid/expired JWT.                                                                       |
| 403 Forbidden             | Authenticated, but RBAC denies the action, or cross-Tenant access attempted.                       |
| 404 Not Found             | Resource does not exist within the caller's Tenant scope.                                          |
| 409 Conflict              | Business-rule conflict (e.g., invalid status transition, duplicate ticket number).                 |
| 422 Unprocessable Entity  | Schema-valid but business-invalid payload (e.g., AMC end_date before start_date).                  |
| 429 Too Many Requests     | Rate limit exceeded (global or per-Tenant).                                                        |
| 500 Internal Server Error | Unhandled server fault; logged with a correlation ID returned to the client for support follow-up. |

## 7.7 API Versioning Strategy

- URL path versioning: /api/v1/..., /api/v2/... — explicit and cache-friendly, avoids ambiguous header-based versioning for a multi-client (web + mobile) platform.
- A version is supported for a minimum deprecation window (target: 6 months) after the next version ships, communicated via a Deprecation response header, to give the mobile app store-review cycle time to update.
- Additive, backward-compatible changes (new optional fields, new endpoints) do not require a version bump; breaking changes (removed/renamed fields, changed semantics) always do.

## 7.8 Sample APIs

### 7.8.1 Login

```
POST /api/v1/auth/login
Content-Type: application/json
```

```
{ "email": "ops.manager@tenantco.com", "password": "*****" }
```

```
200 OK
{
  "success": true,
  "data": {
    "access_token": "eyJhbGciOi...",
    "refresh_token": "9f3e7c1b...",
    "token_type": "bearer",
    "expires_in": 900,
    "user": { "id": "u_123", "name": "Asha Rao", "role": "Operations Manager", "tenant_id": "t_55" }
  }
}
```

### 7.8.2 Customer Management

```
POST /api/v1/customers
{
  "category": "CHS",
  "name": "Sunrise Heights CHS",
  "registration_number": "MH/CHS/2014/0451",
  "contacts": [ { "name": "R. Mehta", "role": "RWA Chairman", "phone": "+91-98xxxxxx10" } ],
  "sites": [ { "address": "Plot 12, Sector 7, Navi Mumbai", "geo_lat": 19.03, "geo_lng": 73.02 } ]
}

GET /api/v1/customers?category=CHS&status=Active&q=sunrise&page=1&page_size=20
```

### 7.8.3 AMC Management

```
POST /api/v1/amc-contracts
{
  "customer_id": "c_881",
  "site_id": "s_204",
  "start_date": "2026-07-01",
  "end_date": "2027-06-30",
  "coverage_type": "comprehensive",
  "visits_included": 4,
  "billing_cycle": "quarterly"
}

GET /api/v1/amc-contracts?status=Expiring%20Soon&expiring_within=30
```

### 7.8.4 Service Tickets

```
POST /api/v1/tickets
{
  "site_id": "s_204",
  "asset_id": "a_3310",
  "category": "Camera Fault",
  "priority": "High",
  "description": "Camera 4 (Gate entrance) showing no live feed since morning."
}

201 Created
{ "success": true, "data": { "id": "T-2026-00822", "status": "Open", "sla_due_at": "2026-06-21T10:00:00Z" } }
```

### 7.8.5 Inventory

```
POST /api/v1/inventory/stock-movements
{
  "item_id": "inv_77",
  "warehouse_id": "wh_van_12",
  "txn_type": "issue",
  "qty": 2,
  "ref_type": "engineer_visit",
  "ref_id": "v_5521"
}
```

```
GET /api/v1/inventory/items?low_stock=true
```

### 7.8.6 Quotations

```
POST /api/v1/quotations
{
  "customer_id": "c_881",
  "lead_id": "l_2090",
  "line_items": [
    { "description": "4MP Dome Camera", "qty": 8, "unit_price": 3200, "tax_pct": 18 },
    { "description": "Installation Labor", "qty": 1, "unit_price": 12000, "tax_pct": 18 }
  ]
}
```

```
POST /api/v1/quotations/{id}/respond
{ "decision": "approved" }
```

### 7.8.7 Invoices

```
POST /api/v1/invoices
{
  "customer_id": "c_881",
  "amc_contract_id": "amc_410",
  "billing_period": "2026-Q3",
  "line_items": [ { "description": "AMC Service Charge - Q3 FY27", "qty": 1, "unit_price": 9000,
  "tax_pct": 18 } ]
}
```

```
GET /api/v1/invoices/{id}/pdf
```

### 7.8.8 Reports

```
POST /api/v1/reports/ticket-turnaround/run
{ "date_from": "2026-04-01", "date_to": "2026-06-30", "site_id": null }
```

```
GET /api/v1/reports/ticket-turnaround/export?format=xlsx&date_from=2026-04-01&date_to=2026-06-30
```

## 8. Security Architecture

Figure 8 summarizes the end-to-end security control chain, from transport security at the edge through to backup/DR. Each control is detailed in the subsections below; together they implement the SRS's security requirements (Sections 4.21 and 5.5).

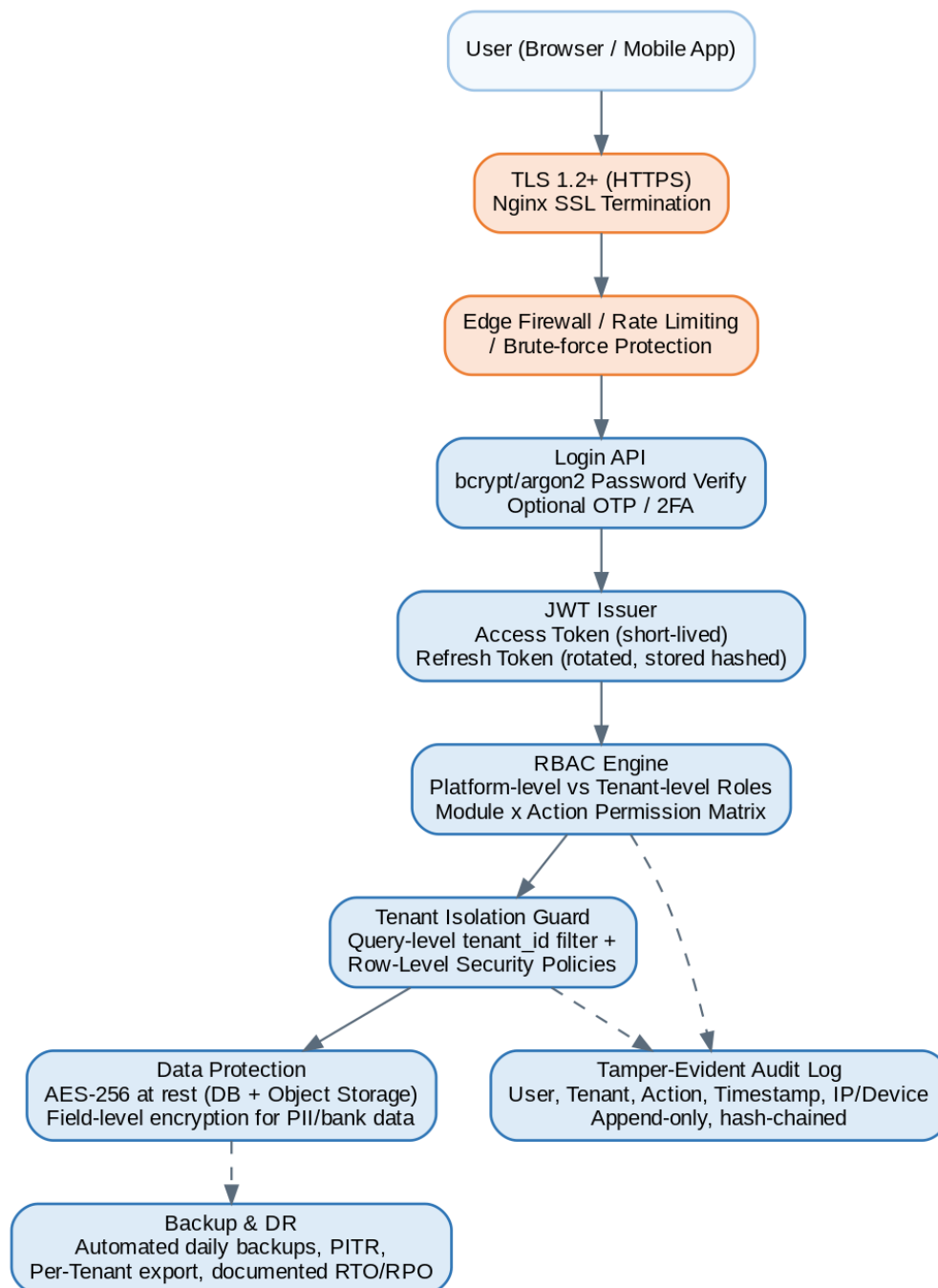


Figure 8: Security Architecture – Control Chain

## 8.1 Authentication & Authorization

- Password-based authentication with argon2/bcrypt hashing (never reversible encryption) and configurable password-strength policy.
- Optional OTP/2FA (SMS or email) per Tenant policy, required for Platform Super Admin accounts by default.
- Account lockout after a configurable number of failed attempts, with progressive backoff.
- Authorization via the two-tier RBAC model (Section 7.3) evaluated on every request after authentication and tenant resolution.

## 8.2 JWT Token Strategy

| Token         | Lifetime            | Storage                                                   | Notes                                                                                         |
|---------------|---------------------|-----------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| Access token  | ~15 minutes         | In memory (web) / secure storage (mobile)                 | Short-lived to limit exposure if leaked.                                                      |
| Refresh token | 7–30 days, rotating | HttpOnly secure cookie (web) / Keychain/Keystore (mobile) | Single-use; rotated and the prior value invalidated on every refresh, detecting token replay. |

- Tokens are signed (RS256/HS256) with a server-held key; the signing key is rotated on a documented schedule and on suspected compromise.
- Tokens embed tenant\_id and role as claims; these claims are the only source of truth for tenant/role context — never a client-supplied header or body field.
- Refresh tokens are revocable server-side (logout, password change, admin-forced session termination, device loss).

## 8.3 Password Encryption

- Passwords are hashed with argon2id (preferred) or bcrypt with a strong work factor; salts are unique per password and managed by the hashing library, not application code.
- Password reset uses a single-use, time-limited token delivered via email/SMS — passwords are never transmitted or displayed in plaintext after initial set.

## 8.4 API Security

- All endpoints require authentication except /auth/login, /auth/refresh, and public health checks.
- Input validation via Pydantic schemas on every request rejects malformed/oversized/unexpected payloads before they reach business logic.
- Per-Tenant and per-IP rate limiting (Section 7.4) mitigates abuse and brute-force attempts.
- CORS is restricted to known frontend origins per environment (Section 12); wildcard origins are never used in UAT/Production.
- All cross-Tenant access attempts are rejected with 403/404 (never leaking existence of another Tenant's resource) and logged to the audit log as a security event.

## 8.5 SSL / TLS Configuration

- TLS 1.2 minimum (TLS 1.3 preferred) enforced at the Nginx edge; plain HTTP requests are redirected to HTTPS.
- Strong cipher suites only; weak/legacy ciphers and protocols (SSLv3, TLS 1.0/1.1) disabled.

- HSTS enabled with a long max-age once the platform's certificate strategy is stable.
- Certificates are issued by a trusted CA (or ACME-based auto-renewal) and monitored for expiry with alerting well ahead of renewal deadlines.

## 8.6 Audit Logs

Every create, update, delete, and approval action across every module writes an entry to a central, append-only audit\_log table, satisfying SRS Section 4.21:

```
audit_log
id, tenant_id, user_id, action (create/update/delete/approve),
entity_type, entity_id, before_json, after_json,
ip_address, device_info, created_at, integrity_hash
```

- integrity\_hash is computed over each row plus the previous row's hash (hash chaining), making undetected tampering with historical entries computationally infeasible — the SRS's "tamper-evident" requirement.
- Audit log is exposed read-only to the Auditor role and Tenant Admins (scoped to their own Tenant) and to the Platform Super Admin (platform-wide, for security investigations).

## 8.7 Role-Based Access Control (RBAC)

Detailed in Section 7.3. Summary: Platform-level roles (Platform Super Admin) vs. Tenant-level roles (Tenant Admin, Sales Executive, Operations Manager, Service Coordinator, Field Technician, Accounts/Billing Staff, Procurement/Vendor Manager, Society/Client Admin, Auditor), each mapped to a (module, action) permission set that Tenant Admins can customize within their own Tenant.

## 8.8 Data Protection Strategy

- **Encryption in transit:** TLS everywhere — client-to-edge, edge-to-application, and application-to-database connections.
- **Encryption at rest:** full-disk/volume encryption for database and object storage; field-level (column) encryption for the most sensitive fields (vendor bank details, customer payment references).
- **Data minimization:** only fields required for AMC/service/billing operations are collected; optional GSTIN for unregistered single-shop clients per SRS Section 4.3.
- **Tenant data export:** each Tenant can request a complete export of its own data (SRS Section 4.21), supporting both operational needs and regulatory data-portability expectations.
- **Compliance posture:** aligned to applicable Indian data-protection and GST invoicing regulations (SRS Section 6.2); legal/compliance review of final data-handling procedures is recommended before go-live.

## 8.9 Backup & Disaster Recovery

| Aspect           | Approach                                                                                                                                                  |
|------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Backup frequency | Automated full daily backups plus continuous WAL archiving for point-in-time recovery.                                                                    |
| Backup retention | Rolling retention window (e.g., 30 daily, 12 monthly) per data-retention policy, stored in a separate region/availability zone from the primary database. |

| Aspect              | Approach                                                                                                                                                           |
|---------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Per-Tenant export   | On-demand export of a single Tenant's data, supporting Tenant offboarding or compliance requests without restoring the whole platform.                             |
| Recovery objectives | Target RPO (Recovery Point Objective) and RTO (Recovery Time Objective) are defined and tested during UAT (Section 12.2) and documented in the operations runbook. |
| DR testing          | Scheduled restore drills validate that backups are actually restorable, not just taken.                                                                            |

## 9. Notification Architecture

All notifications — regardless of triggering module — flow through one Notification Engine (Section 5.15), which resolves the Tenant-specific template and recipient(s) for the event, then dispatches to the appropriate channel. This keeps template management, delivery tracking, and retry/dedup logic in a single place rather than duplicated per module.

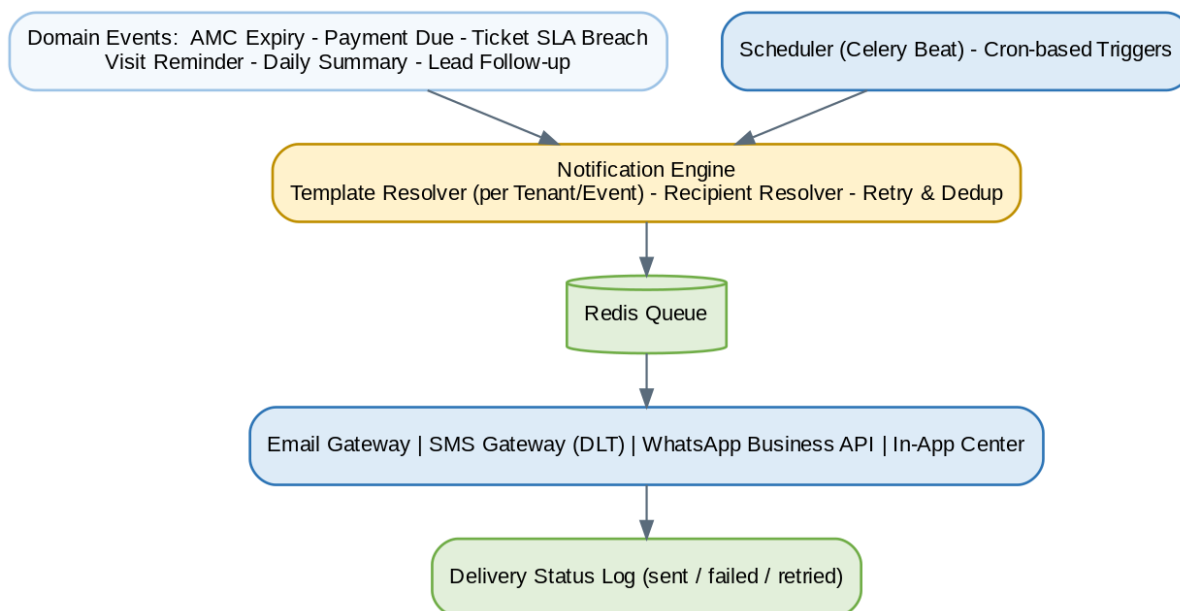


Figure 9: Notification Architecture

### 9.1 Channel Design

| Channel  | Use Case                                                                       | Provider Pattern                                                  |
|----------|--------------------------------------------------------------------------------|-------------------------------------------------------------------|
| Email    | Quotations, invoices, receipts, detailed reminders, daily summaries.           | SMTP relay or transactional email API (e.g., SES/SendGrid-class). |
| SMS      | Time-sensitive, short alerts: OTP, SLA breach, visit reminder.                 | DLT-registered SMS gateway (India compliance).                    |
| WhatsApp | Customer-preferred channel for invoices, quotation links, visit confirmations. | WhatsApp Business API provider.                                   |
| In-App   | Internal staff: assignments, escalations, approvals pending.                   | WebSocket push or short-poll to /api/v1/notifications/in-app.     |

### 9.2 Event Catalog

| Event            | Trigger                                                      | Channel(s)                   | Recipient(s)                                    |
|------------------|--------------------------------------------------------------|------------------------------|-------------------------------------------------|
| AMC Expiry Alert | Celery Beat scan finds AMC contracts expiring within N days. | Email, SMS, WhatsApp, In-App | Customer + Sales Executive + Operations Manager |
| Payment Reminder | Invoice ageing crosses a                                     | Email, SMS,                  | Customer +                                      |



| Event                              | Trigger                                             | Channel(s)              | Recipient(s)                             |
|------------------------------------|-----------------------------------------------------|-------------------------|------------------------------------------|
|                                    | configured threshold (e.g., 30/60/90 days).         | WhatsApp                | Accounts/Billing Staff                   |
| Ticket Escalation                  | SLA timer breached or about to breach.              | SMS, In-App, Email      | Service Coordinator + Operations Manager |
| Daily Summary                      | Scheduled end-of-day job per Tenant.                | Email, In-App           | Tenant Admin + Operations Manager        |
| Lead Follow-up Reminder            | lead_activity.next_action_at reached.               | In-App, SMS             | Assigned Sales Executive                 |
| Visit Reminder                     | Scheduled preventive-maintenance visit approaching. | SMS, WhatsApp           | Customer + Field Technician              |
| Quotation Sent / Viewed / Expiring | Quotation status change or approaching expiry.      | Email, WhatsApp, In-App | Customer + Sales Executive               |

### 9.3 Template & Delivery Design

- **Per-Tenant templates:** notification\_template rows are keyed by (tenant\_id, event\_type, channel), so each Tenant can customize wording/branding without code changes (SRS Section 4.15).
- **Idempotent dispatch:** each enqueued notification carries a dedup key (e.g., amc\_contract\_id + event\_type + date) so retried jobs cannot double-send the same alert.
- **Retry with backoff:** transient provider failures (Email/SMS/WhatsApp gateway timeouts) are retried with exponential backoff up to a bounded attempt count, then marked failed and surfaced on a delivery-failure dashboard.
- **Delivery tracking:** notification\_log records every attempt's outcome, enabling support staff to confirm whether a customer was actually notified.
- **Quiet hours / throttling:** non-urgent channels (Email, Daily Summary) respect a configurable quiet-hours window per Tenant; urgent channels (SLA escalation) are not throttled.

## 10. Mobile Application Architecture

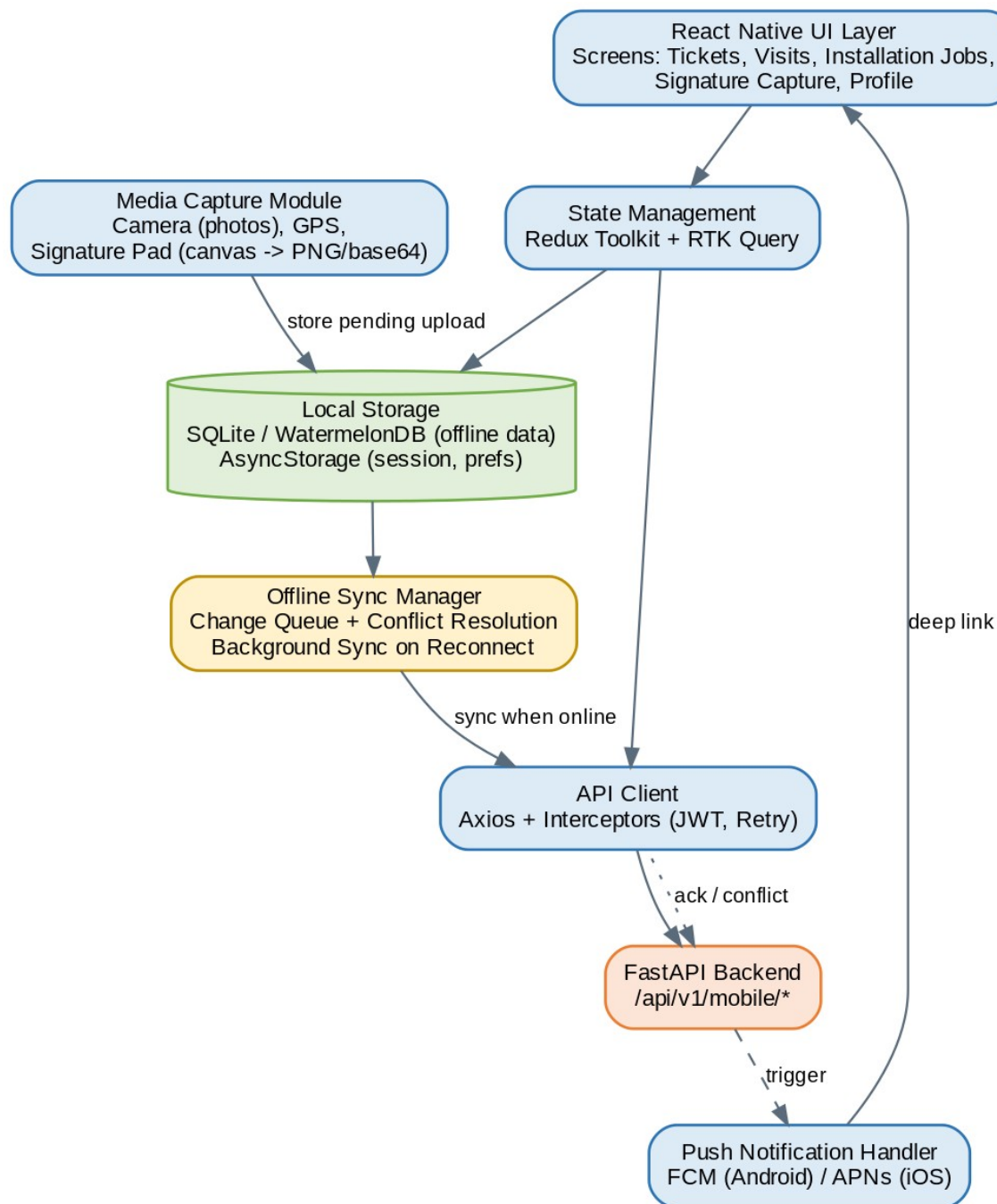


Figure 10: Mobile Application Architecture (React Native)

### 10.1 React Native Architecture

- **UI layer:** screen components for assigned tickets, installation jobs, visit detail, signature capture, and profile, built on the same component conventions as the web apps where feasible (shared design tokens, not shared code).

- **State management:** Redux Toolkit mirrors the web app's pattern (Section 4.1) for consistency, with an added persisted slice for offline queue state.
- **Navigation:** stack + tab navigation separating "Today's Visits", "All Tickets", and "Profile/Sync Status".
- **API client:** Axios with interceptors for JWT attachment and silent refresh, identical pattern to web (Section 4.1).

## 10.2 Offline Sync Strategy

- All technician-relevant data (assigned tickets, visits, installation jobs) for the current day is pre-fetched and cached locally on login/refresh, so the app is usable the moment connectivity drops.
- Every mutating action (status update, parts consumed, photo/signature capture) is first written to a local change queue, then optimistically reflected in the UI.
- A background Sync Manager flushes the queue to POST /api/v1/mobile/sync whenever connectivity is available, batching multiple changes into one round trip.
- Conflict resolution: server is authoritative for status transitions (last-write-wins is rejected if the server-side status has already moved past the client's assumed prior state); the app surfaces a conflict banner with the server's current state rather than silently overwriting it.
- Sync status (last synced at, pending item count) is always visible to the technician so unresolved offline work is never silently lost.

## 10.3 Local Storage Design

| Store                   | Used For                                                                           | Technology                                                                                 |
|-------------------------|------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| Structured offline data | Tickets, visits, installation jobs, inventory snapshot relevant to the technician. | SQLite (via WatermelonDB or equivalent ORM)                                                |
| Pending change queue    | Queued mutations awaiting sync.                                                    | SQLite table, processed FIFO with retry                                                    |
| Session / preferences   | JWT (encrypted), last sync timestamp, UI preferences.                              | Keychain (iOS) / Keystore (Android) for tokens; AsyncStorage for non-sensitive preferences |
| Pending media           | Captured photos/signatures awaiting upload.                                        | Device file system, referenced by the change queue until confirmed uploaded                |

## 10.4 Media Upload Architecture

- Photos and signature images are captured at a bounded resolution/compression to balance evidentiary quality against upload size on poor connections.
- Media is queued for upload independently of the record it's attached to: the visit/ticket update can sync immediately while a large photo upload retries in the background, and the server treats the record as complete only once all required media has arrived.
- Uploads go directly to object storage via a signed upload URL issued by the API (avoiding routing large binaries through the API application servers), with the resulting key written back to the relevant record.

## 10.5 Digital Signature Capture Workflow

6. Technician completes the visit/installation checklist in the app.
7. App presents either a signature pad (finger/stylus capture rendered to a PNG) or an OTP-confirmation screen, per Tenant configuration.
8. On signature: the canvas is exported to a PNG and queued for upload via the Media Upload Architecture above, linked to the visit/installation record.
9. On OTP: the app calls the OTP-send endpoint (customer receives via SMS/WhatsApp), the customer reads it to the technician (or enters it on a handed-over device), and the app submits it for verification.
10. Once signature/OTP is confirmed, the visit/installation record is marked complete and triggers downstream actions (inventory deduction, AMC creation on handover, invoice eligibility).

## 10.6 Push Notification Architecture

- Device registers its push token against the authenticated user via POST `/api/v1/mobile/devices/register` on login and on token refresh (FCM for Android, APNs for iOS).
- The same Notification Engine (Section 9) that drives Email/SMS/WhatsApp also drives push: a 'push' channel entry in `notification_template/notification_log` keeps delivery tracking consistent across all channels.
- Push payloads carry a deep-link reference (e.g., visit ID) so tapping the notification opens the relevant screen directly, even if the app was backgrounded.
- Push tokens are invalidated and removed on logout or when a refresh-token revocation occurs (e.g., device reported lost), preventing notifications to a de-provisioned device.

## 11. Reporting & Analytics Architecture

### 11.1 Dashboard Architecture

Dashboards (Section 5.17) never compute KPIs live against the transactional database on page load. A background worker job recomputes `kpi_snapshot` rows on a schedule (e.g., every 5–15 minutes, or on the relevant write event for fast-changing KPIs), and the dashboard API simply reads the latest snapshot for the current user's role and Tenant. This keeps dashboard load times within the SRS's 3-second target (Section 5.1) regardless of how much historical data a Tenant has accumulated.

| Role                              | Dashboard Focus                                                                       |
|-----------------------------------|---------------------------------------------------------------------------------------|
| Tenant Admin / Operations Manager | Lead pipeline, AMC renewal pipeline, open tickets & SLA compliance, revenue summary.  |
| Sales Executive                   | My leads by stage, my quotations pending response, follow-ups due today.              |
| Service Coordinator               | Today's dispatch board, unassigned tickets, technician availability.                  |
| Field Technician                  | My visits today, my completed visits this week.                                       |
| Accounts/Billing Staff            | Outstanding dues by ageing bucket, today's collections, overdue invoices.             |
| Procurement/Vendor Manager        | Low-stock alerts, open purchase orders, vendor payable balances.                      |
| Platform Super Admin              | Tenant count by status, usage vs. plan limits, platform-wide error/health indicators. |

### 11.2 Reporting Engine Design

- **Report catalog:** `report_definition` rows describe each available report (name, parameter schema, underlying query/view), exposed to a user filtered by their role's permissions.
- **Execution path:** synchronous execution for small/fast reports; large date-range or cross-Tenant (platform-admin) reports are executed asynchronously via the worker tier with the result made available for download once complete, avoiding long-held HTTP connections.
- **Read replica isolation:** all report execution reads from the PostgreSQL read replica (Section 6.7), never the primary, so heavy reporting cannot degrade transactional performance.

### 11.3 KPI Framework

| KPI                     | Definition                                                        | Module Source             |
|-------------------------|-------------------------------------------------------------------|---------------------------|
| Lead Conversion Rate    | Converted leads ÷ total qualified leads, by period.               | Lead Management           |
| Installation Turnaround | Days from approved quotation to handover.                         | Sales Order Management    |
| Ticket Turnaround Time  | Time from ticket open to resolved, by priority.                   | Service Ticket Management |
| SLA Compliance %        | Tickets resolved within SLA ÷ total tickets.                      | Service Ticket Management |
| Technician Productivity | Visits completed per technician per day; average resolution time. | Engineer Visit Management |

| KPI                        | Definition                                        | Module Source        |
|----------------------------|---------------------------------------------------|----------------------|
| Inventory Consumption Rate | Stock issued per period by item/category.         | Inventory Management |
| Vendor Performance         | On-time delivery %, return/quality incident rate. | Vendor Management    |
| Revenue by Customer/Site   | Invoiced amount by customer/site/period.          | Invoice Management   |
| AMC Renewal Rate           | Renewed contracts ÷ contracts due for renewal.    | AMC Management       |

## 11.4 Data Aggregation Strategy

- Pre-aggregation (kpi\_snapshot, materialized views) for dashboard-grade, frequently-viewed metrics.
- On-demand aggregation (parameterized SQL views/queries against the read replica) for ad hoc, filtered reports.
- Tenant scoping is applied identically to aggregation queries as to transactional queries — RLS and the tenant\_id filter apply uniformly (Section 2.4); only Platform Super Admin reports intentionally aggregate across Tenants.

## 11.5 Export to Excel / PDF

- Excel export via a server-side XLSX generation library, applied to the same parameterized report result used for on-screen display, ensuring exported and displayed numbers always match.
- PDF export reuses the Document/PDF Generation Service (Section 4.4) used for quotations/invoices, applying a report-specific layout template.
- Large exports are generated asynchronously by the worker tier and delivered via a download link (and optionally emailed), avoiding timeouts on large date ranges.

## 12. Deployment Architecture

### 12.1 Development Environment

- Single-node Docker Compose stack (Nginx, FastAPI, Celery worker, PostgreSQL, Redis) for local development, mirroring production service topology at a smaller scale.
- Seed/fixture data covering a sample multi-Tenant setup (at least two Tenants) so tenant-isolation behavior is visible and testable from day one of development.
- Feature branches deploy to ephemeral preview environments (or a shared Dev environment) via the CI/CD pipeline (Section 12.4) for early integration testing.

### 12.2 UAT Environment

- Production-equivalent topology at reduced capacity (Section 12.3), refreshed from a sanitized production-like dataset or representative seed data.
- Used for QA functional/non-functional sign-off, multi-tenant isolation testing (Section 14), and client/stakeholder review prior to production release.
- Deployments to UAT are automatic on merge to the develop branch (Section 12.4); UAT is the gating environment before a production release is tagged.

### 12.3 Production Environment

- Minimum two web nodes (Nginx + FastAPI + Celery worker containers) behind a cloud load balancer for both capacity and availability (Section 13.1).
- Managed PostgreSQL primary with at least one read replica; managed or self-hosted Redis cluster for cache/session/queue.
- Object storage (S3-compatible) for all binary content, with versioning enabled for accidental-overwrite protection.
- All inbound traffic over HTTPS only; SSH/administrative access restricted by IP allow-list and key-based authentication.

### 12.4 Docker Deployment Architecture

| Image  | Contains                                                                             | Scaling Unit                                              |
|--------|--------------------------------------------------------------------------------------|-----------------------------------------------------------|
| nginx  | Reverse proxy config, TLS certs (mounted/secret-injected), static asset cache rules. | Per web node                                              |
| api    | FastAPI application + Gunicorn/Uvicorn workers.                                      | Horizontally replicated                                   |
| worker | Celery worker process(es) for background tasks.                                      | Horizontally replicated, independent of API replica count |
| beat   | Celery Beat scheduler (single active instance) for cron-style triggers.              | Singleton (leader-elected if HA required)                 |

Each image is built once in CI (Section 16.1) and promoted unmodified from Dev → UAT → Production by tag, never rebuilt per environment — eliminating environment drift as a source of "works in UAT, fails in Production" defects.

## 12.5 CI/CD Pipeline

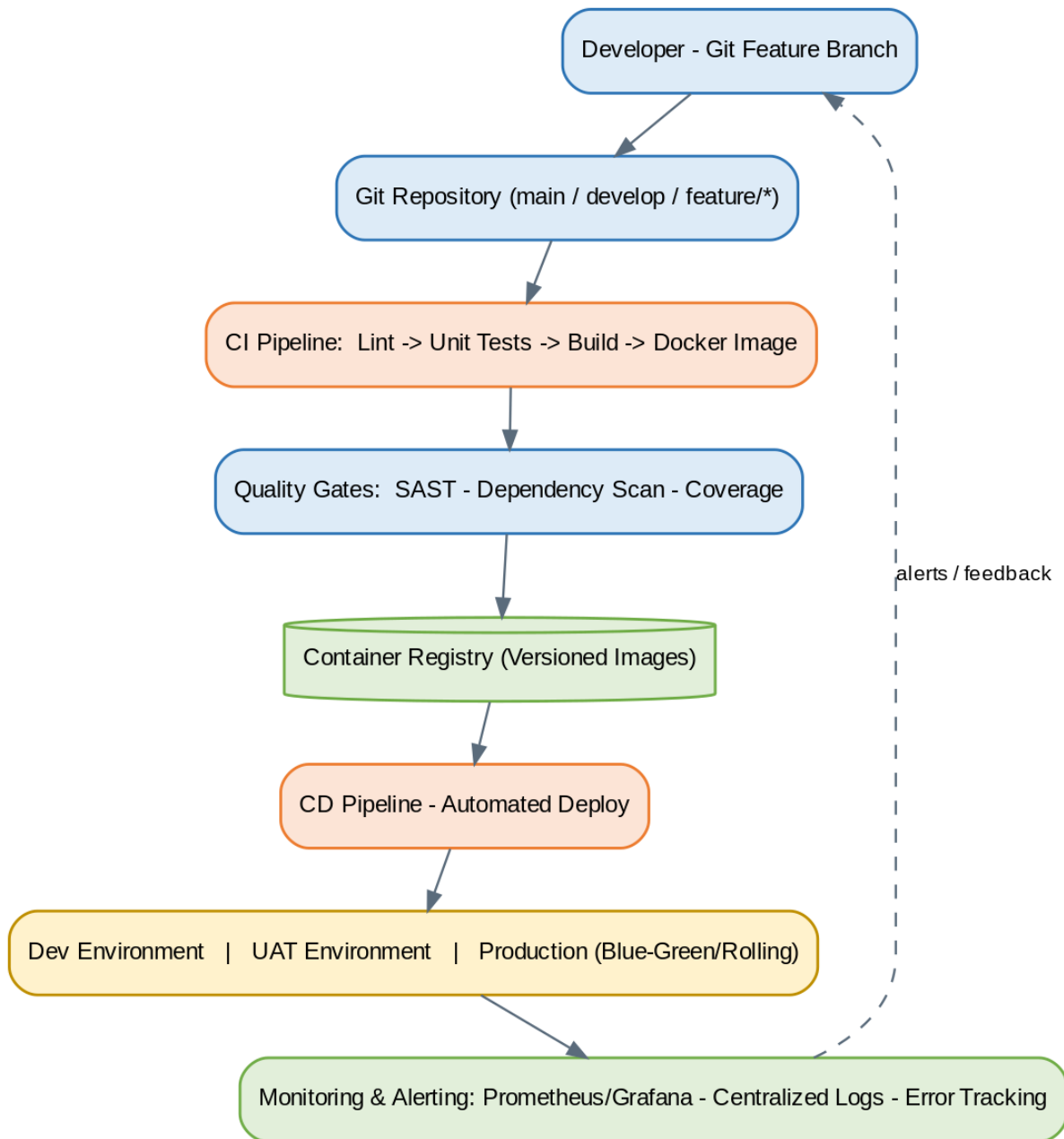


Figure 11: CI/CD & DevOps Pipeline

11. Developer pushes to a feature branch; pull request opens against develop.
12. CI runs lint, static type checks, unit tests, and builds Docker images for changed services.
13. Security/quality gates run: dependency vulnerability scan, container image scan, test-coverage threshold check.



14. On merge to develop, the pipeline pushes versioned images to the container registry and auto-deploys to UAT.
15. On a tagged release (e.g., v1.4.0) and manual approval, the same images are deployed to Production via a rolling or blue-green strategy.
16. Post-deploy smoke tests verify health-check endpoints before routing full traffic to the new version; automatic rollback to the previous tag on smoke-test failure.

## 12.6 Monitoring & Logging Architecture

- **Application metrics:** request rate, latency percentiles, and error rate per endpoint, scraped from FastAPI via a metrics endpoint (Prometheus-compatible) and visualized on dashboards (Grafana-class).
- **Infrastructure metrics:** CPU, memory, disk, and container health per node, with threshold-based alerting.
- **Centralized logging:** structured (JSON) logs from API and worker containers shipped to a central log store, correlated by a request ID that flows from Nginx through every layer down to the audit log entry.
- **Error tracking:** unhandled exceptions are captured with stack trace and request context in a dedicated error-tracking tool, deduplicated by error signature for triage.
- **Uptime monitoring:** external synthetic checks against the public login and health-check endpoints, independent of the internal monitoring stack, to validate the 99.5% uptime target (SRS Section 5.4) from the outside in.

## 13. Scalability & Performance Architecture

### 13.1 Horizontal Scaling Strategy

- Stateless FastAPI containers: session state lives in the JWT and Redis, not in-process, so any container can serve any request — new replicas are added behind Nginx/load balancer purely by capacity, with no sticky-session requirement.
- Celery workers scale independently of API replicas, sized to background-job volume (notification fan-out, recurring billing) rather than request volume.
- New Tenant onboarding (SRS Section 5.2) requires no architectural change or downtime — it is purely a new row in tenant plus associated config, made possible by the shared-schema multi-tenancy model (Section 2.4).

### 13.2 Vertical Scaling Strategy

- Used as a first lever for the database tier (CPU/RAM increase on the PostgreSQL primary) before horizontal read-replica scaling is needed, since vertical scaling requires no application change.
- Container resource limits (CPU/memory requests/limits) are tuned per service based on observed load before defaulting to horizontal scale-out.

### 13.3 Database Scaling

- Read replica(s) absorb reporting/dashboard load (Section 6.7); additional replicas can be added as Tenant count and reporting volume grow.
- Table partitioning (audit\_log, notification\_log, stock\_ledger) bounds index size and vacuum cost as data accumulates (Section 6.7).
- Connection pooling (PgBouncer) decouples the number of application containers from the number of physical database connections.
- If a single PostgreSQL cluster's write throughput is ever exceeded by Tenant growth, the shared-schema design leaves Tenant-based sharding (grouping Tenants across multiple PostgreSQL clusters behind a routing layer) as a viable future path without a full data-model rewrite.

### 13.4 Caching Strategy

| Cache                       | Content                                    | Invalidation                                  |
|-----------------------------|--------------------------------------------|-----------------------------------------------|
| Tenant config/branding      | Theme, feature flags, plan limits.         | On Tenant profile/plan update                 |
| RBAC permission matrix      | Role → permission mapping.                 | On role/permission change                     |
| KPI snapshots               | Dashboard-ready aggregates (Section 11.1). | Scheduled recompute / on relevant write event |
| Session/rate-limit counters | Per-user and per-Tenant request counters.  | Time-window expiry (sliding/fixed window)     |

## 13.5 Load Balancing

- Cloud load balancer (or DNS-based) distributes traffic across Nginx nodes; Nginx in turn load-balances across FastAPI container replicas (least-connections).
- Health checks at both layers remove unhealthy nodes/containers from rotation automatically.

## 13.6 Performance Optimization Techniques

- Index coverage for every hot query path (Section 6.6), validated via CI-time EXPLAIN checks.
- N+1 query prevention via SQLAlchemy eager-loading conventions (Section 15.1).
- Pagination enforced on every list endpoint (no unbounded result sets).
- Response payload trimming: list endpoints return summary fields; full detail is fetched only on a record's detail view.
- Async I/O in FastAPI for all DB/HTTP-bound operations to maximize concurrent request handling per container.
- Gzip/Brotli compression at the Nginx layer for all text-based responses.

## 14. Non-Functional Requirements

These requirements are carried forward and made architecturally concrete from the source SRS (Section 5), with the implementing mechanism named alongside each.

### 14.1 Performance

| Requirement                                                               | Architectural Mechanism                                                                                             |
|---------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| Standard pages/dashboards load within 3 seconds under normal load.        | KPI pre-aggregation (Section 11.1), Redis caching (Section 13.4), indexed query paths (Section 6.6), async FastAPI. |
| Support at least 200 concurrent users per deployment without degradation. | Horizontally scaled stateless API tier (Section 13.1) + load balancing (Section 13.5).                              |

### 14.2 Availability

| Requirement                                                                                | Architectural Mechanism                                                                                                             |
|--------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| 99.5% uptime target for web and customer portal during business hours, across all Tenants. | Multi-node deployment with load balancing and health-check-driven failover (Section 13.1/13.5); monitoring/alerting (Section 12.6). |
| Mobile app supports offline operation with reliable sync-on-reconnect.                     | Offline-first local storage and sync manager (Section 10.2–10.3).                                                                   |

### 14.3 Reliability

- PostgreSQL primary/replica with streaming replication and automated backups (Sections 6.7, 8.9) protect against data loss.
- Idempotent write APIs (Section 7.1) and a durable Celery queue (Redis-backed) ensure background jobs (notifications, billing) are not silently dropped on transient failure.
- Rolling/blue-green deployments with automated smoke tests and rollback (Section 12.5) prevent a bad release from taking down the platform.

### 14.4 Security

Fully detailed in Section 8. Summary: JWT-based authentication, two-tier RBAC, TLS 1.2+ everywhere, encryption at rest for sensitive fields, tamper-evident audit logging, and enforced Tenant data isolation at both the application and database (Row-Level Security) layers.

### 14.5 Maintainability

- Modular architecture (Section 2.3, Section 5): each business module owns its entities, service logic, and API routes, allowing independent updates across all Tenants without a full-platform redeploy of unrelated modules.
- Per-Tenant configuration (notification templates, SLA rules, tax rates, branding) is data-driven, not code-driven — changing it never requires a code deployment.

- Generated OpenAPI documentation (Section 7.1) keeps the API contract self-documenting as modules evolve.

## 14.6 Extensibility

- New business modules can be added as new FastAPI routers + services + repositories without touching existing modules, following the conventions in Section 4.2.
- Integration-ready hooks (payment gateway, NVR/DVR systems, future white-label apps — SRS Section 7, Future Enhancements) are designed as pluggable adapters behind existing service interfaces rather than requiring architectural rework.
- The shared-schema multi-tenancy model leaves a documented path to Tenant-based database sharding if a future Tenant's scale requires it (Section 13.3), without a data-model rewrite.

## 14.7 Usability

- Consistent component library (Material UI / Ant Design) across all three web surfaces minimizes the learning curve when staff move between modules.
- Customer portal is responsive and requires no app installation, accessible directly from a mobile browser (SRS Section 5.6).
- Mobile app UI is optimized for one-handed, outdoor, on-site use by field technicians (large touch targets, minimal required typing, camera/GPS-first workflows).

## 14.8 Compliance

- GST-compliant invoice generation and Indian currency/date/number formatting by default (SRS Section 5.9).
- DLT-registered SMS gateway usage for the Indian market (Section 9.1).
- Data handling aligned to applicable Indian data-protection requirements (SRS Section 6.2); formal legal/compliance sign-off is recommended ahead of production go-live.
- Browser/OS compatibility: latest two major versions of Chrome, Edge, and Safari (web); Android 9+ and iOS 14+ (mobile) per SRS Section 5.7.

## 15. Development Standards

These standards apply to all contributors (in-house team, contractors, or vendor partners) and are a baseline condition for code merge approval.

### 15.1 Coding Standards

#### Backend (Python / FastAPI)

- PEP 8 compliant; enforced automatically via Black (formatter) and Ruff/Flake8 (linter) in CI.
- Type hints mandatory on all function signatures; validated with mypy.
- Pydantic v2 models for all request/response schemas — no raw dict payloads.
- One router per module (Section 4.2); business logic lives in the service layer, never in route handlers.
- All DB access goes through the repository layer — no raw SQL or ORM calls inside service/route code.

#### Frontend (React / TypeScript)

- Strict TypeScript mode enabled (``strict: true``); no implicit ``any``.
- ESLint + Prettier enforced pre-commit (via Husky) and in CI.
- Functional components with hooks only; class components are not permitted in new code.
- Redux Toolkit slices per feature domain; no direct state mutation outside reducers.
- API calls isolated to a dedicated ``services/`` layer using the shared Axios instance — components never call Axios directly.

#### Mobile (React Native)

- Shares the same TypeScript/ESLint/Prettier conventions as the web frontend.
- Platform-specific code isolated behind ``ios.tsx`` / ``android.tsx`` file conventions, not inline ``Platform.OS`` branching where avoidable.

### 15.2 Git Branching Strategy

A trunk-based variant of Gitflow is used, matching the three environments in Section 12:

| Branch    | Purpose                                                | Deploys To                  |
|-----------|--------------------------------------------------------|-----------------------------|
| main      | Always production-ready; protected, merge via PR only. | Production (on tag/release) |
| develop   | Integration branch for completed features.             | UAT (auto on merge)         |
| feature/* | One branch per ticket/story, branched from develop.    | Dev (on push)               |
| release/* | Cut from develop for release stabilization/hardening.  | UAT                         |
| hotfix/*  | Branched from main for urgent production fixes.        | Production (fast-track)     |

- Every PR requires at least one approving review and a passing CI pipeline (Section 16.1) before merge.
- Commit messages follow Conventional Commits (`feat:`, `fix:`, `chore:`, `refactor:`, ...) to drive automated changelog generation.
- No direct pushes to `main` or `develop`; both are branch-protected.

### 15.3 API Development Standards

- Every endpoint follows the REST and envelope conventions defined in Section 7.1/7.5 — no exceptions without an explicit, reviewed deviation.
- Every endpoint must declare its required role(s)/permission(s) explicitly via a FastAPI dependency — there is no 'implicitly open' endpoint.
- Every new or changed endpoint must update its OpenAPI schema/docstring in the same PR; the generated API docs are treated as the contract.
- Breaking changes require a new API version (Section 7.7), never an in-place contract change on a published version.
- All list endpoints must support pagination; unbounded result sets are not permitted.

### 15.4 Database Design Standards

- Every table includes `id` (UUID), `tenant\_id`, `created\_at`, `updated\_at`, and `is\_deleted` (soft delete) unless explicitly justified otherwise (Section 6.1).
- All foreign keys are indexed; all `tenant\_id` columns are indexed (Section 6.6).
- Schema changes are made exclusively through versioned Alembic migrations — manual production schema edits are prohibited.
- Every migration must be reversible (a working `downgrade()`), reviewed, and tested against a staging copy of representative data volume before production rollout.
- Naming convention: snake\_case for tables/columns, singular table names avoided in favor of plural (e.g., `service\_tickets`), and consistent suffixes (`\_id`, `\_at`, `\_by`).

### 15.5 Documentation Standards

- API contract: auto-generated OpenAPI/Swagger, kept current as part of the Definition of Done for any API change.
- Architecture: this TAD is the source of truth; material architectural changes require an update to the relevant section and version bump (Document Control table) before merge.
- Code-level: docstrings required on all service-layer functions and non-trivial business logic; inline comments reserved for explaining 'why', not 'what'.
- Runbooks: each production-supporting service (API, worker, scheduler) has an operational runbook covering startup, health checks, common failure modes, and escalation path.

### 15.6 Testing Standards

| Layer              | Tooling | Minimum Expectation                                                                                 |
|--------------------|---------|-----------------------------------------------------------------------------------------------------|
| Backend unit tests | pytest  | Service-layer business logic covered; target ≥80% coverage on core modules (AMC, Tickets, Billing). |

| Layer                     | Tooling                                     | Minimum Expectation                                                                                 |
|---------------------------|---------------------------------------------|-----------------------------------------------------------------------------------------------------|
| Backend integration tests | pytest + test DB (containerized PostgreSQL) | Every API endpoint has at least one happy-path and one authorization-failure test.                  |
| Frontend unit tests       | Jest + React Testing Library                | Critical components (forms, ticket workflow, dashboard widgets) covered.                            |
| End-to-end tests          | Playwright/Cypress                          | Core user journeys: login, create AMC, raise ticket, complete visit, generate invoice.              |
| Mobile tests              | Jest + Detox (or equivalent)                | Offline sync flow and digital signature capture flow covered as critical paths.                     |
| Load/performance tests    | Locust/k6                                   | Run pre-production against the 200-concurrent-user NFR target (Section 14.1) before major releases. |

CI blocks merge to develop/main on any failing test suite (Section 16.1); QA sign-off on the UAT environment is required before a release branch is promoted to production.



## 16. DevOps & Monitoring

This section operationalizes the deployment topology of Section 12 and the CI/CD pipeline shown in Figure 11. It defines how the platform is built, shipped, observed, and recovered on an ongoing basis.

### 16.1 CI/CD Process

17. Developer opens a PR from a feature branch into develop (Section 15.2); CI triggers automatically.
  18. CI stage runs: lint (ESLint/Ruff), type-check (tsc/mypy), unit tests (Jest/pytest), and a dependency vulnerability scan.
  19. On merge to develop, CI builds Docker images for the API, worker, and frontend, tags them with the commit SHA, and pushes to the container registry.
  20. CD auto-deploys the develop build to the UAT environment; smoke tests run against UAT post-deploy.
  21. On approval and merge of a release branch to main (tagged with a semantic version), CD builds production images and deploys via rolling/blue-green strategy (Section 12.5).
  22. Post-deploy health checks gate traffic cutover; automatic rollback to the previous image tag triggers if health checks fail.
- Pipeline implemented in GitHub Actions/GitLab CI (tool-agnostic; either fits the stack).
  - Secrets (DB credentials, JWT signing keys, gateway API keys) are injected via the CI/CD secrets store — never committed to the repository.
  - Database migrations (Alembic) run as an explicit pipeline step before the new application version receives traffic.

### 16.2 Container Management

| Concern         | Approach                                                                                                                                          |
|-----------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| Image strategy  | Multi-stage Dockerfiles per service (Section 12.4); slim base images (python:3.12-slim, node:20-alpine) to minimize attack surface and pull time. |
| Registry        | Private container registry (cloud provider registry or self-hosted Harbor) with image vulnerability scanning on push.                             |
| Orchestration   | Docker Compose for Dev/UAT; Docker Swarm or a managed Kubernetes service for Production at scale (Section 13.1).                                  |
| Configuration   | Environment-specific config via env files/secret stores per environment — never baked into the image.                                             |
| Image lifecycle | Tagged by commit SHA + semantic version; old/unused image versions pruned by automated retention policy.                                          |

## 16.3 Application Monitoring

- Infrastructure & container metrics (CPU, memory, disk, network) collected via Prometheus, visualized in Grafana dashboards per environment.
- Application-level metrics: request rate, latency (p50/p95/p99), error rate per endpoint, Celery queue depth/lag.
- Uptime/synthetic checks against critical endpoints (login, dashboard, ticket creation) from outside the cluster, feeding the 99.5% availability target (Section 14.2).
- Alerting (via Alertmanager/PagerDuty/Opsgenie-equivalent) on error-rate spikes, latency breaches, queue backlog, disk pressure, and certificate expiry — routed to the on-call engineer.

## 16.4 Error Tracking

- Centralized error tracking (Sentry or equivalent) integrated into both the FastAPI backend and the React/React Native frontends.
- Each captured error includes request context, Tenant ID, authenticated user (where available), and a correlation/request ID for cross-referencing with logs.
- Error grouping and release-tagging let the team see whether a given release introduced a regression, supporting fast rollback decisions.

## 16.5 Log Management

- Structured JSON logging from the API and worker processes; every log line carries request ID, tenant\_id, user\_id, and timestamp.
- Logs shipped via a log forwarder (Fluent Bit/Vector) to a centralized store (ELK stack or a managed logging service) with role-restricted access.
- Nginx access/error logs and PostgreSQL slow-query logs are ingested into the same pipeline for unified correlation during incident investigation.
- Log retention policy: hot/searchable for 30 days, archived (cold storage) for a longer compliance-driven retention period, consistent with the audit log retention in Section 8.6.

## 16.6 Backup Strategy

| Asset                                          | Backup Approach                                                                | Frequency / Retention                         |
|------------------------------------------------|--------------------------------------------------------------------------------|-----------------------------------------------|
| PostgreSQL database                            | Automated full snapshot + continuous WAL archiving for point-in-time recovery. | Daily full + continuous WAL; 30-day retention |
| Object storage (photos, signatures, documents) | Cross-region replicated bucket / versioning enabled.                           | Continuous (versioned); lifecycle-managed     |
| Application configuration & secrets            | Encrypted backup of the secrets store/config repo.                             | On every change                               |
| Container images                               | Retained in registry with version tags.                                        | Indefinite for production-tagged releases     |

Restore procedures are documented as part of the operational runbooks (Section 15.5) and rehearsed periodically via a disaster-recovery drill, consistent with the RPO/RTO targets defined in Section 8.9.

## 17. Project Deliverables

The table below lists each contracted deliverable and cross-references exactly where it is satisfied within this document, together with its standalone file equivalent provided alongside this TAD.

| #  | Deliverable                              | Location in this Document                             | Standalone File Provided                                               |
|----|------------------------------------------|-------------------------------------------------------|------------------------------------------------------------------------|
| 1  | Complete Technical Architecture Document | This document, Sections 1–17                          | CCTV_AMC_SaaS_Technical_Architecture_Document.docx                     |
| 2  | High-Level Architecture Diagram          | Section 2.2, Figure 1                                 | 01_high_level.png                                                      |
| 3  | Low-Level Architecture Diagram           | Section 2.3, Figure 3 (Component-Based Architecture)  | 03_component.png                                                       |
| 4  | Database ER Diagram                      | Section 6.2, Figure 6                                 | 06_er_diagram.png                                                      |
| 5  | API Architecture Diagram                 | Section 7.2, Figure 7                                 | 07_api_architecture.png                                                |
| 6  | Security Architecture Diagram            | Section 8.1, Figure 8                                 | 08_security_architecture.png                                           |
| 7  | Deployment Architecture Diagram          | Section 2.5, Figure 5 (Physical Deployment)           | 05_physical_deployment.png                                             |
| 8  | DevOps Architecture Diagram              | Section 12.5, Figure 11 (CI/CD Pipeline)              | 11_cicd_devops.png                                                     |
| 9  | Mobile Application Architecture Diagram  | Section 10.1, Figure 10                               | 10_mobile_architecture.png                                             |
| 10 | Technology Stack Recommendation Report   | Section 3.9 (Technology Stack Recommendation Summary) | Included in main document; see also Section 3.1–3.8 for full rationale |

### 17.1 Supplementary Diagrams

Two additional diagrams were produced in support of the above deliverables and are included both in-document and as standalone files, as they are referenced repeatedly across multiple sections:

| Diagram                                       | Location              | Standalone File                  |
|-----------------------------------------------|-----------------------|----------------------------------|
| Layered Architecture                          | Section 2.1, Figure 2 | 02_layered.png                   |
| Logical / Multi-Tenant Isolation Architecture | Section 2.4, Figure 4 | 04_logical_multitenant.png       |
| Notification Architecture                     | Section 9.1, Figure 9 | 09_notification_architecture.png |

### 17.2 Acceptance Notes

- All diagrams are provided as high-resolution PNG files (rendered at 150–200 DPI source) so they remain legible when zoomed or reprinted independently of this document.

- This TAD should be reviewed and formally signed off by the client/stakeholders identified in the Document Control table before the Development Team begins Sprint 1 implementation.
- Any deviation from the mandated technology stack, module list, or multi-tenant data model discovered during implementation should be raised as a documented change request against this TAD, not silently implemented.